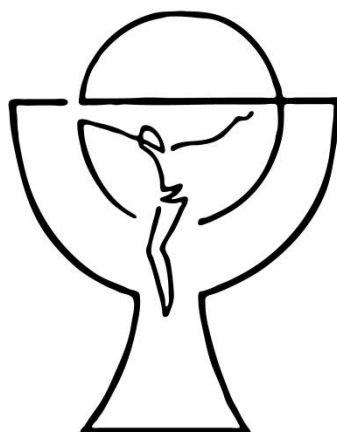




SZENT ISTVÁN KATOLIKUS
TECHNIKUM ÉS GIMNÁZIUM

DAKOS JÁNOS BENCE
FODOR GERGŐ
SEDLÁK PATRIK ANDRÁS
HEBLE

(Healthy Body Lifestyle Entertainment)

SZOFTVERFEJLESZTÉS ÉS -TESZTELÉS
VIZSGAREMEK

Sátoraljaújhely, 2025

Tartalomjegyzék

I. Bevezetés.....	4
II. A projekt jellemzése	5
III. Használt technológiák és eszközök.....	6
Kódíráshoz használt.....	6
Egyéb fejlesztői eszközök.....	7
IV. FELHASZNÁLÓI DOKUMENTÁCIÓ	9
Rendszerkövetelmények	9
Minimális	9
Ajánlott.....	9
Alkalmazás funkciói:	9
Főoldal.....	10
Rólunk és Kapcsolat.....	11
Edzés és Teljesítmények (kijelentkezett felhasználó).....	12
Ingyenes tartalmak.....	12
Bejelentkezés.....	14
Regisztráció.....	14
Jelszó helyreállítás.....	15
Bejelentkezett állapot.....	16
Edzés.....	17
Teljesítmények.....	18
Profil.....	19
V. FEJLESZTŐI DOKUMENTÁCIÓ	21
Telepítés és beindítás menete	21
Backend komponens dokumentációja	30
Bevezetés	30
Adatbázis felépítése	33
Kontrollerek.....	36
Végpontok dokumentációja.....	40
Szolgáltatások	48
Authenticator.....	51

Frontend komponens dokumentációja.....	52
Bevezetés	52
app.routes.ts és app.config.ts.....	53
services	53
interfaces	55
guards	55
komponensek	57
Admin felület	63
VI. TESZTELÉS	68
Backend tesztelés.....	68
Frontend tesztelés.....	68
Teszteset 1. – Sikeres regisztráció	68
Teszteset 2. – Sikertelen jelszóhelyreállítás.....	69
Teszteset 3. – Sikeres gyakorlathozzáadás és teljesítmény elérés, elvégzett gyakorlatok	69
Terheléses tesztelés.....	70
VII. Továbbfejlesztési lehetőségek.....	71
VIII. Irodalomjegyzék	73

I. Bevezetés

A projekten dolgozó mindhárom személy fontosnak találja a mozgást, viszont más-más formában. A mozgás azon része, ami a szabadtéri mozgásformákra építene, az az elmúlt években teljesen átalakult. Az emberek igényei megváltoztak. Egyre több időt töltenek az okoseszközeik előtt. Ezek a szociális változások adtak motivációt a kis csapatunknak arra, hogy létrehozzuk a következő projektet, amelyben találkozik a mozgás, az egészséges életmód és a szórakoztatás. Így motiválva másokat a szabadban végzett mozgásra, hogy életük mindennapi részévé váljon ismét.

A projekt neve első látásra értelmetlennek tűnhet, de van valós mögöttes jelentése. A HeBLE egy mozaikszó, amelynek felbontásából kaphatjuk meg a teljes nevét, ami Healthy Body Lifestyle Entertainment. A kulcsszavak a projekt lényegét tartalmazzák. Azaz, hogy mit is szeretnénk nyújtani a projektünkkel. Az angol szavakból megkaphatjuk a Healthy Body, mint egészséges test, a Lifestyle, mint életmód és végül az Entertainment, mint szórakoztatás szavakat. Így egyszerre összekötve a kellemeset a hasznossal.

Tehát projektünk célja az emberek motiválása, hogy minél több szabadtéri mozgást végezzenek. Erre egy webalkalmazással akarjuk felhívni a figyelmüket, amelyben különböző mozgásos gyakorlatok teljesítésével előrehaladást érhetnek el a felhasználók. Ezt az előrehaladást pont úgy mérjük, mint egy videójátékba. Azaz tapasztalatpontokba és szintekbe. Az elvégzett gyakorlatokért a felhasználó tapasztalatpontot kap és bizonyos megszerzett tapasztalatpont után szintet léphet. A gyakorlatok adottak, ezek nem változnak. Ami változik, az az, hogy egy adott szint elérése érdekében mennyi gyakorlatot kell elvégezni, vagyis a szükséges tapasztalatpontok száma folyamatosan növekszik. Így a felhasználóknak egyre több időt kell eltölteniük a mozgással. A felhasználók emellett teljesítményeket érhetnek el, ezzel is maguk előtt tartva különböző célkitűzéseket. Ezeket a célkitűzéseket is folyamatosan tudják követni és láthatják, hogy miből mennyi gyakorlatot végeztek el eddig. Ezekhez a tartalmakhoz csak regisztrált és bejelentkezett felhasználók férhetnek hozzá, míg egyébként regisztráció nélkül is érhetőek el ingyenes tartalmak, amelyek különböző témákkal foglalkoznak.

II. A projekt jellemzése

Tehát egy olyan alkalmazás készül, ami az egészséges életmódra ösztönözi a felhasználókat. Így mozgást igénylő és végző felhasználóknak készül, elsősorban azoknak, akik most kezdenek el ismerkedni a mozgás világával.

Fontos kihangsúlyozni, hogy a webalkalmazás becsült alapon működik jelen állás szerint, azaz a felhasználók a saját becsületük alapján jelölhetik meg elvégzettként a gyakorlatokat.

A szoftver futtatható lesz különböző okos eszközök böngészőjén keresztül. Célunk, hogy egy felhasználóbarát, azaz könnyen kezelhető, átlátható és teljesen reszponzív felületet hozzunk létre, ami újszerű és játékos módon közelíti meg az eddigi „nyers” mozgást. Mindenki számára elérhető ingyenes tartalmak mellett a regisztrációhoz és bejelentkezéshez köthető funkciókkal van ellátva a szoftver.

Röviden a technológiai háttéről: a program fejlesztői környezete a Visual Studio Code volt, a frontend keretrendszer az Angular, míg a backend esetében a szerverfuttatási környezet a Node.js, az úgyszintén backendhez tartozó adatbázis pedig MySQL. A fejlesztés előtt megbeszéltük és közösen felosztottuk, hogy ki mivel szeretne foglalkozni. A megbeszéltek alapján Dakos János Bence és Fodor Gergő a frontendet készítette el, Sedlák Patrik András, pedig a backendet.

III. Használt technológiák és eszközök

A projektünk elkészítéséhez a vizsgakövetelményben által megkövetelt modern technológiákat és egyéb eszközöket használtunk.

Kódíráshoz használt

- **Frontend:**
 - A fejlesztői környezet a **Visual Studio Code**, amelyet az iskolai tanulmányaink során végig használtunk, így ezt ismertük a legjobban. A keretrendszer, pedig az **Angular** volt, annak is a **19-es** verziója.
- **Frontend technológiák és eszközök:**
 - **HTML5**, amely segítségével a webalkalmazásunk struktúráját határoztuk meg. **CSS3**, amely segítségével különböző stílusbeállításokat rendelhettünk az alkalmazásunkhoz. A **TypeScript** (v.5.6.3), pedig egy JavaScript alapú programozási nyelv. Emellett a projekt elkészítéshez használtunk különböző **Bootstrap ikonokat és komponenseket**. Ilyen felhasznált komponens a *navbar, footer, cards, toasts, alerts, forms*. A bootstrap-et emellett a reszponzív megjelenés teljes biztosításáért is használtuk.
 - A Visual Studio Code-ba hozzáadott bővítményeket (**Extensions**) használtunk. Ilyen például a **Prettier**, amit kódformázáshoz használtunk, hogy a kód átláthatóbb és olvashatóbb legyen. Ezzel megfelelve a clean code egyik alapelvének.
- **Backend:**
 - A fejlesztői környezet továbbra is a Visual Studio Code, a projektmunkához használt szerverfuttatási környezet a **Node.js** volt. Ebben így a mások által létrehozott és fejlesztett csomagokat használtunk a projektünk elkészítéséhez.
 - Különböző bővítmények felhasználására itt is sor került. Ilyen a **Babel JavaScript**, ami segített a szintaktikai hibák megtalálásában, míg az **ERD Editor** segítségével adatbáziskapcsolatokat lehetett vizualizálni. Ezek a Visual Studio Code fejlesztői által is hitelesített bővítmények.

Csomagok:

- @faker-js/faker -- Randomizált adatok felvitelére az adatbázisban,
 - bcrypt -- Jelszó hashelő könyvtár,
 - bcryptjs -- Optimalizált bcrypt könyvtár JavaScripthez 0 függőséggel,
 - cors -- Csomag ahhoz, hogy biztonságosan emeljünk be erőforrásokat a webalkalmazásunkra, egy külső forrásból (domain-ről). Ilyen külső erőforrás lehet például, egy kép, egy betűcsalád, szkript stb.,
 - dotenv -- A környezeti változók kezelésére,
 - express -- Backend JS keretrendszer REAST API készítésre,
 - express-session -- egy session middleware létrehozása beállításokkal,
 - faker -- ^6.6.6,
 - jsonwebtoken -- Könyvtár a token kezelésre,
 - mariadb -- A dialektus értelmezésére,
 - nodemon -- Folyamatosan futó szerveret fenn tartó csomag,
 - sequelize -- Adatbázis modellek legenerálása és kezelése.
- **Adatbázis:** MySQL (MariaDB) [XAMPP]. A tanulmányaink alatt a XAMPP-ot használtunk, így ezt ismertük a legjobban és ezt találtuk megfelelőnek.

Egyéb fejlesztői eszközök

Csoportmunkát segítő szoftverek:

- **Trello** egy projekttervezéshez és -szervezéshez használt eszköz, így ez egy megfelelő lehetőséget nyújtott nekünk a projektünk során elvégzendő tevékenységekhez. Elkészítettünk egy előzetes tervet, amelyben megfogalmaztuk, hogy mit és hogyan szeretnénk elkészíteni. Felosztottuk a feladatokat, hogy ki mit fog elkészíteni a projektben. Folyamatosan frissítettük a különböző elérhető dokumentumokat. Külön jelöltük az elkészítésre váró feladatokat, megjelöltük azokat, amelyeket már elkezdtünk, illetve külön listákban jelöltük a csapattagok által elvégzett feladatokat.
- **Git és GitHub** a projekt végzése során létrejött dokumentumok, fájlok és kódok tárolására használtuk, a verziókezelésnek köszönhetően folyamatosan nyomon tudtuk követni a változásokat. Ezeknek köszönhetően könnyedén bármikor és

bárhonnan el tudtuk érni a munkánkat. A projekt elvégzése során végig egy main branchet (fő ágat) használtunk.

- Ha bármi egyéb kommunikációt kellett folytatnunk otthon, akkor **Discordot** és **Messengert** használtunk. Ezeken a platformokon gyorsan meg tudunk beszélni mindent, amit szerettünk volna.

Dokumentációkészhez használt eszközök:

- **Microsoft Word** (v16), amit értelemszerűen a dokumentációkészítéshez használtunk.

Képtervezéshez és szerkesztéshez használt szoftverek:

- **Background Remover-t** (remove.bg) használtunk egyes felhasznált képek háttérének eltüntetéséhez. Az **Adobe Photoshop** és a **Canva** a logó és a banner elkészítéséhez használt szoftverek. Az alap elképzelést, hogy mit is szeretnék a projektünk logóján és a bannerén látni azt közösen terveztük meg, majd Dakos János Bence készítette el ezeket vizuálisan. Az logó és banner az alábbiakban megtekinthető, ezek az elkészült és végleges emblémák.



IV. FELHASZNÁLÓI DOKUMENTÁCIÓ

Rendszerkövetelmények

Az alábbiakban felsoroljuk a projektünk minimális és ajánlott rendszerkövetelményeit, ezeket érdemes ellenőrizni a webalkalmazás elindítása előtt a legjobb felhasználói élményért. Fontos megjegyeznünk, hogy a projektünk elindításához aktív és stabil internetkapcsolatra lesz szükség. Illetve minimális szabad merevlemez-területre, optimális esetben 2GB minimum.

Minimális

- Processzor: Intel Core 3 vagy AMD Ryzen 3
- Memória: 4GB RAM
- Videókártya: Integrált megfelel
- Operációs rendszer: Windows 10/11, Linux, macOS
- Böngésző: Google Chrome, Opera, Microsoft Edge, Safari

Ajánlott

- Processzor: Intel Core 3 vagy AMD Ryzen 3
- Memória: 8GB RAM
- Videókártya: GeForce GTX 1050 vagy AMD RX550
- Operációs rendszer: Windows 10/11, Linux, macOS
- Böngésző: Google Chrome, Opera, Microsoft Edge, Safari

Alkalmazás funkciói:

A webalkalmazásunkat a felhasználók az okos eszközökön, mint például asztali számítógép vagy laptop és egy általuk választott böngészőn érhetik el.

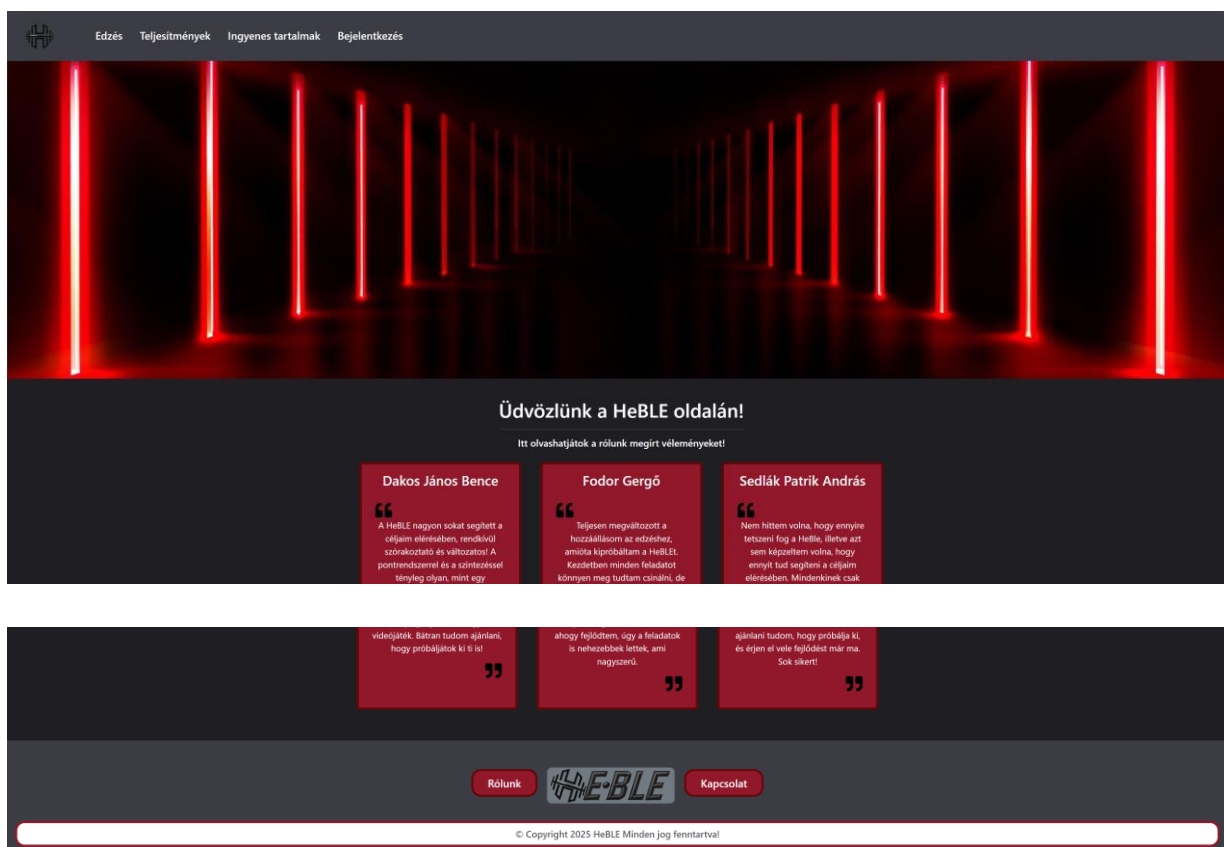
A projekt a megfelelő programok telepítése és beállítása után futtatható és ez által elérhető a <http://localhost:4200/> -on keresztül.

A felhasználókat az URL meglátogatása után a nyitó oldal, azaz a Főoldal fogadja.

Főoldal

Ahogy említettem az előbb a felhasználót ez fogadja, így minden esetben ez az alap kiindulási pontunk bármiféle tevékenység elvégése előtt. A piros és a fekete különböző árnyalataiba öltözött megjelenés fogadja az webalkalmazásra látogatókat, ez a szín tökéletesen passzolt az egész webalkalmazásunk témájához.

A lap tetején egy állandó navigációs menüsor található, amelyben a felhasználó szabadon navigálhat. Ezek a menüpontok a következők: Edzés, Teljesítmények, Ingyenes tartalmak és a Bejelentkezés. Megjegyzésként szolgáljon, hogy a HeBLE logójára kattintva a felhasználót azonnal a Főoldalra navigálja bárhol is legyen a felhasználó. A navigációs sáv alatt egy képet helyeztünk el, majd a rólunk megírt vélemények kártyák jelennek meg. Úgyszintén állandó elemként megjelenő lábléc, pedig az oldal legalján helyezkedik el. Amelyben megtalálható a Rólunk és a Kapcsolat navigációs menüpont.



Rólunk és Kapcsolat

Ahogy említettem ez a kettő a láblécben helyezkedik el. Míg a Rólunk részen a felhasználók elolvashatják a legfontosabb információkat rólunk, mint például, hogy kik vagyunk? Mi ez a név? Mit akarunk nyújtani? Majd a végén a csatlakozásra motiváljuk őket. Ezt, azért gondoltuk fontosnak elhelyezni a webalkalmazásunkon, mert az átlag felhasználó nem fog hozzáférni ehhez a dokumentációhoz, amelyben ugye az elején már érintettük ezeket a kérdéseket és megmagyaráztunk mindent, de így a felhasználók is megérthetik ezeket. A Kapcsolat, ahogy annak nevek, úgy a leírása is pontosan azt tartalmazza, amit ígér. Egy email címet, amelyen keresztül a felhasználók felvehetik velünk a kapcsolatot, ha bármi kérdésük vagy észrevételük akadt. Emellett egyéb közösségi oldalakon való elérhetőségeink láthatóak.



Rólunk

A csapatunk számára rendkívül fontos az egészséges életmód és a testmozgás. Ez motivált minket arra, hogy létrehozzuk a HeBLE-t. **Mit is jelent a HeBLE?** Az angol szavakból álló rövidítés felbontható a következőket jelenti: *Healthy Body*, mint egészséges test, *Lifestyle* mint életmód és *E* mint *Entertainment*, azaz szórakoztatás. Tehát pont azt, amit nyújtani szeretnénk a felhasználóinknak.

Célunk

A HeBLE célja tehát az emberek motiválása a testmozgásra szórakoztató, izgalmas megoldásokkal ötvözve. **Ez hogyan is működik?** A felhasználók mindennap különböző mozgási feladatokat kapnak, amelyek idővel nehezebbek lesznek. Az elvégzésükért pontokat kapnak, amelyekkel szintet léphetnek. Különböző teljesítményeket és jutalmakat oldhatnak fel a felhasználók, ezzel is motiválva őket a mindennapokban.

Csatlakozz!

Ha meggyőztünk a rövid bemutatkozásunkkal akkor regisztrálj és próbáld ki! Éj el fejlődést és változást meg az életed a HeBLE-vel!



Kapcsolat

Ha szeretnél velünk a HeBLE csapatával kapcsolatba lépni és segítséget kérni valamilyen vagy esetlegesen te szeretnél számunkra tanácsot adni abban, hogy mi miben fejlődünk, netán a véleményed szeretnéd kifejtetni akkor írd nekünk egy emailt bátran!

Az alábbi email címen léphetsz velünk kapcsolatba

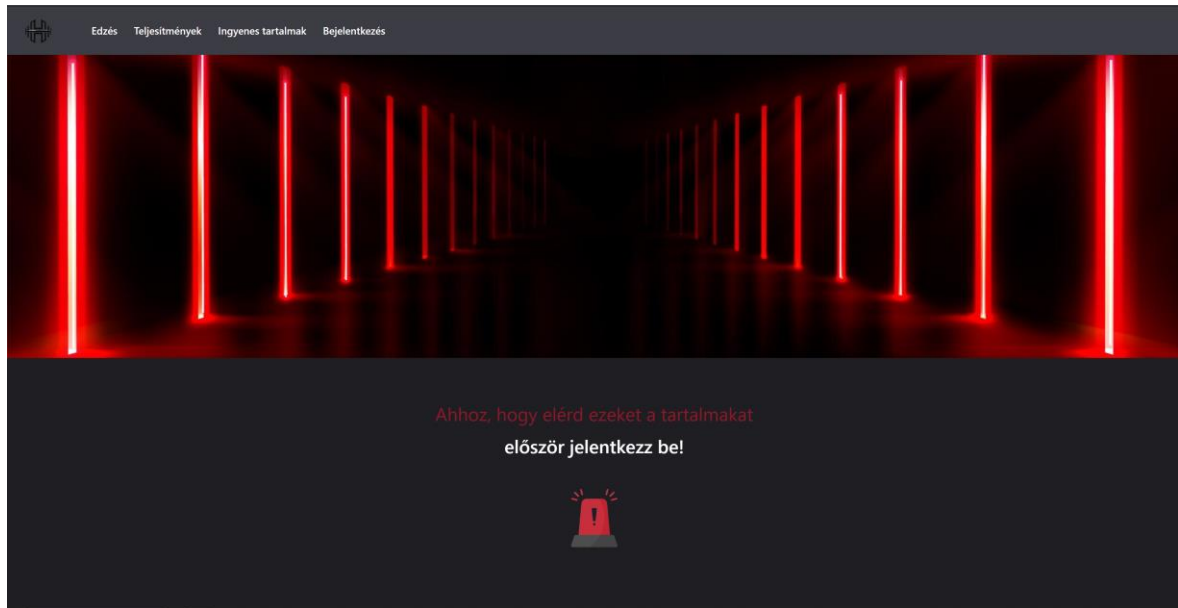
HeBLEstrength@gmail.com

Megtalálhatsz minket a különböző közösségi oldalakon is!

[f](#) [@](#) [X](#)

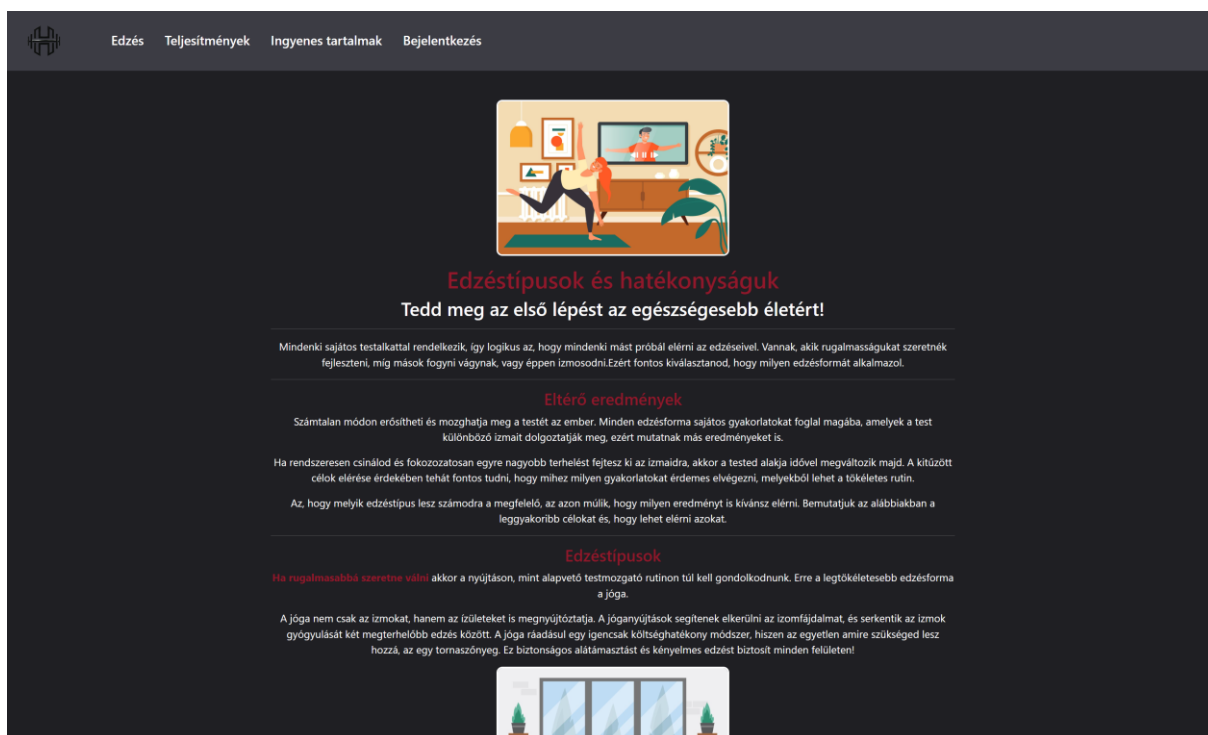
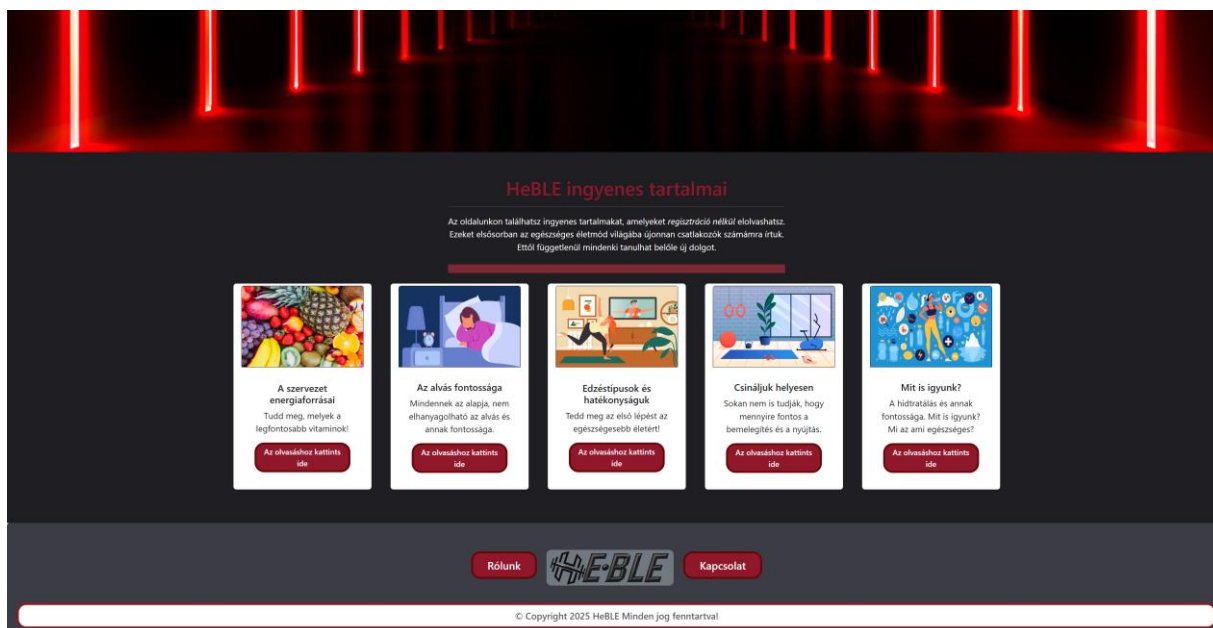
Edzés és Teljesítmények (kijelentkezett felhasználó)

Amikor a felhasználó nincs bejelentkezve, akkor nem fér hozzá bizonyos tartalmakhoz. Ilyen az Edzés és a Teljesítmények. Ezek ugyanis regisztrációhoz és bejelentkezett állapothoz vannak kötve. A felhasználót először váratlanul érheti, amikor a képen látható piros figyelmeztető jel fogadja, de amikor elolvassa a felette álló üzenete, akkor azonnal megértheti, hogy ezt miért nem teheti ezt meg jelenleg.



Ingyenes tartalmak

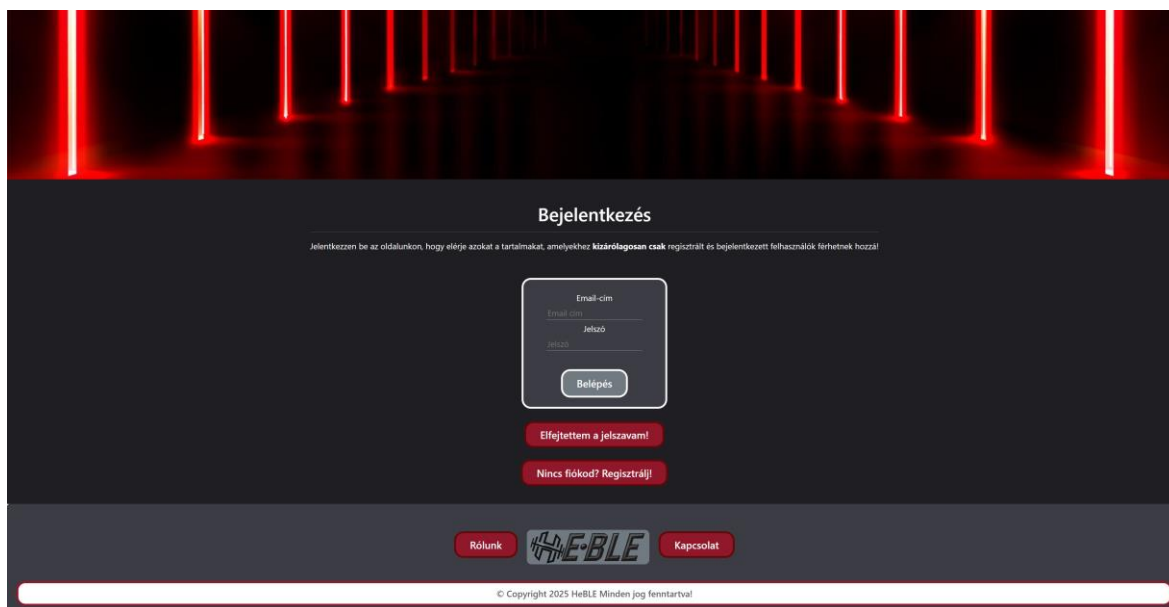
Az ingyenes tartalmak résznél a felhasználó azt találja, amit a név ígér. Azaz ingyenes tartalmakat, amelyek attól ingyenesek, hogy a felhasználók ezeket regisztráció nélkül szabadon olvashatják. Ezek lényegre törően és nagyon kezdőbarát módon vannak megfogalmazva. A felhasználók olyan témákban érhetnek el tartalmakat, mint például a vitaminok, az alvás fontossága, különböző edzéstípusok, a bemelegítés és nyújtás és a folyadékbevitel.



Egy kép az egyik ingyenes tartalomból. Minden egyes ingyenes tartalom alján található egy „Vissza az előző oldalra” gomb, amivel visszatérhetnek az ingyenes tartalmakhoz.

Bejelentkezés

A bejelentkezésnél a fiókunkhoz tartozó email cím és jelszó megadása után, és a Belépés gomb segítségével be tudunk jelentkezni. A Belépés gomb mindaddig szürke (ezáltal meg nem nyomható) marad, ameddig az összes mezőt ki nem tölti a felhasználó. Amennyiben bármilyen adat hibás, így a bejelentkezés sikertelen akkor arról egy felugró értesítés tájékoztat, illetve a sikeres bejelentkezésről is értesít. A sikeres bejelentkezés követően át irányít automatikusan az eddig elérhetetlen (be nem jelentkezett felhasználóknak) Edzéshez. Ha egy felhasználó nem rendelkezik regisztrált fiókkal, akkor ezt könnyedén orvosolhatja a „Nincs még fiókod? Regisztráld!” Gomb segítségével, ami átirányítja a felhasználót a Regisztrációhoz. Amennyiben valaki elfelejtette a jelszavát az a felette lévő „Elfelejtetem a jelszavam!” gomb segítségével megkezdheti a fiók visszaszerzési folyamatát.



Bejelentkezés

Jelentkezzen be az oldalunkon, hogy elérje azokat a tartalmakat, amelyekhez kizárólagosan csak regisztrált és bejelentkezett felhasználók férhetnek hozzá!

Email cím

Jelszó

Belépés

Elfelejtetem a jelszavam!

Nincs fiókod? Regisztráld!

Rólunk **HeBLE** Kapcsolat

© Copyright 2025 HeBLE Minden jog fenntartva!

Regisztráció

Ha a felhasználó nem rendelkezik fiókkal és, így átnavigálta magát a Bejelentkezéstől a Regisztrációhoz, úgy egyből megkezdheti a regisztrációt. A Regisztráció gomb mindaddig szürke (ezáltal meg nem nyomható) marad, ameddig az összes mezőt ki nem tölti a felhasználó. A regisztráció során a felhasználótól a következő adatokat kéri: Keresztnév, Vezetéknév, Email cím, Jelszó és Biztonsági Jelszó. Az, hogy mi is a biztonsági jelszó és, hogy mire való, arról a felhasználó egy rövid magyarázót olvashat a regisztrációs űrlap alatt. Röviden a biztonsági jelszó egy egyszerű jelszó, amire akkor van szükség, ha a felhasználó elfelejtené a jelszavát. Így célszerű, hogy ez minél egyszerűbb és ez által

könnyen megjegyezhető legyen! Ha a felhasználó kitöltötte a kért adatokat, akkor a „Regisztráció” gomb segítségével elvégezheti a regisztrációt. Amennyiben bármilyen adat hibás akkor arról felugró értesítés tájékoztat minket, illetve a sikeres regisztráció esetén is értesítést add vissza. Sikeres regisztráció után automatikusan átnavigál minket a Bejelentkezéshez. Ha a felhasználó véletlen módján keveredett volna ide, akkor a biztonsági jelszó magyarázat után található „Van már fiókod? Lépj be!” gomb segítségével könnyedén visszatérhet a Bejelentkezéshez.

Jelszó helyreállítás

Ha egy felhasználó elfelejtené a jelszavát, így a bejelentkezésre képtelene. Akkor a jelszavát könnyedén helyreállíthatja. A Bejelentkezésnél található „Elfelejtettem a jelszavam” gomb segítségével átnavigálhat a Fiók helyreállításához. A felületen a felhasználó megkezdheti a jelszavának helyreállítását. Ehhez a felhasználói fiókhoz tartozó email-címre és biztonsági jelszóra van szükség. Amennyiben a felhasználó ezekkel az adatokkal nem rendelkezik, akkor a helyreállítás nem kezdhető meg. Az „Elküldés” gomb mindaddig szürke (ezáltal meg nem nyomható) marad, ameddig az összes mezőt ki nem tölti a felhasználó. Amennyiben bármilyen adat hibás akkor arról egy felugró értesítés tájékoztat minket, illetve a sikeres elküldés esetén is értesítést add vissza. Sikeres Fiók helyreállítás után automatikusan átnavigál minket a következő szakaszára a folyamatnak. Ahol meg kell adnunk egy új jelszót. Az új jelszó beírása után az „Elküldés” gomb itt is elérhetővé válik. Ha sikeresen megadtuk az új jelszót akkor erről egy felugró értesítés

tájékoztat minket, majd egy rövid idő után automatikusan átnavigál minket a Bejelentkezéshez. Ha a felhasználó netán véletlen keveredett ide volna ide, akkor a „Vissza a Bejelentkezéshez!” gomb segítségével gyorsan vissza navigálhat a Bejelentkezéshez.

Fiók helyreállítás

Elfelejtetted a jelszavad? Van egy jó hírünk! Az e-mailed és biztonsági jelszó segítségével visszaállíthatod a fiókid!

Email-cím

Biztonsági jelszó

Elküldés

Véletlen tévedtél ide?

Vissza a Bejelentkezéshez!

Rólunk HE-BLE Kapcsolat

© Copyright 2025 HeBLE Minden jog fenntartva!

Fiók helyreállítás

Elfelejtetted a jelszavad? Van egy jó hírünk! Az e-mailed és biztonsági jelszó segítségével visszaállíthatod a fiókid!

Új jelszó

Új jelszó

Elküldés

Véletlen tévedtél ide?

Vissza a Bejelentkezéshez!

Rólunk HE-BLE Kapcsolat

© Copyright 2025 HeBLE Minden jog fenntartva!

Bejelentkezett állapot

Sikeres bejelentkezést követően egy rövid idő után automatikusan átnavigál minket az Edzéshez. Ez a tartalom, ahogy fentebb is részleteztük csak a regisztrált és a bejelentkezett felhasználók érhetik el. Ahogyan a Teljesítmények és a Profil navigációs menüpont is mostantól válik elérhetővé a felhasználóknak. A Bejelentkezés helyét átveszi a Kijelentkezés opció, amelynek segítségével a felhasználó könnyedén kiléphet a fiókjából.

Kijelentkezést követően az előbb felsorolt tartalmak értelemszerűen ismét elérhetetlenné válnak.

[Edzés](#)[Teljesítmények](#)[Ingyenes tartalmak](#)[Profil](#)[Kijelentkezés](#)

Edzés

Az Edzésnél a felhasználó először egy rövid három sorból álló bemutatkozóval találkozik, majd alatta rögtön megtekintheti az előrehaladását. Az előrehaladást tapasztalatpontban és szintben mérjük. A felhasználó először egyes szintűként kezd, pont úgy, mint egy videójátékban. A felhasználó jelenlegi szintje alatt láthatja, hogy mennyi tapasztalatpont szükséges a következő szintig. Ez kezdetben 100 pont, a jelenlegi tapasztalatpont pedig 0. A felhasználó tapasztalatpontot úgy szerezhet, hogy különböző gyakorlatok végez el. Ilyen gyakorlat lehet jelenleg a fekvőtámasz, a húzódzkodás, a felülés, a guggolás és a futás. Az elvégzett gyakorlatokat a felhasználó a saját becsülete szerint jelöli meg, hogy miből mennyit végzett el. Ez, azért is fontos, mert ez így nyer értelmét a mozgás. A tisztességes munka kifizeti magát, így érhet el fejlődést a fizikumában is. Az elvégzett gyakorlatok „Beküldés gombja” mindaddig szürke (ezáltal meg nem nyomható) marad, ameddig a felhasználó az összes rendelkezésre álló mezőt ki nem tölti. A felhasználó, ha az adott gyakorlatból aktuálisan nem végzet egy gyakorlatot sem, de fel akarja tölteni akkor azt nyugodtan, megteheti úgy, hogy abba a mezőbe nullát ír be. A sikertelen és sikeres gyakorlatrögzítésről és értesítést kap a felhasználó. Sikeres gyakorlatrögzítés után az oldal frissít egyet és a felhasználó egyből láthatja az előrehaladás változását. Szintenként a következő szintig szükséges tapasztalatpontok mennyisége növekszik, így a felhasználó egyre több és több időt kell majd mozgásra fordítania és több gyakorlatot kell majd elvégezni.

HeBLE Edzés

Próbáld ki még ma a HeBLE szórakoztató, egyszerű és játékos megoldását az edzésre!
 Teljesíthet feladatokat, majd a feladatok elvégzése után gyűjthet be a tapasztalalpontokat és még szintet!
 Érd el különböző teljesítményeket és jutalmakat!

Az alábbiakban megtekintheted az előrehaladásod! Láthatod, hogy épp hányas szintű vagy, mennyi az aktuális tapasztalalpontod és mennyi kell a következő szintig.

Előrehaladás

Tapasztalalpont: 0 pont

Szint: 1

Szükséges tapasztalalpont a következő szintig: 100 pont

Elérhető gyakorlatok

Add meg az elvégzett gyakorlatok mennyiségét számban!

Fekvőtámasz:

Fekvőtámaszok száma

Húzózkodás:

Húzózkodások száma

Felülés:

Felülések száma

Guggolás:

Guggolások száma

Futás:

Futás mennyisége (km-h)

Beküldés

Teljesítmények

A Teljesítményeknél a felhasználó megtekintheti, hogy milyen teljesítményeket érhet el az előrehaladás során. A teljesítményeket különböző osztályokba soroljuk aszerint, hogy mennyire nehéz elérni őket. Amelyeket a trófeák segítségével ábrázoljuk. Minden teljesítmény rendelkezik egy névvel és rövid leírással. A felhasználók ezeket a teljesítményeket maguk előtt tarthatják, mint egyféle célkitűzés, ezzel motivációt nyújtva.

HeBLE Teljesítmények

Ezen az oldalon megnézheted, hogy milyen különböző teljesítmények elérhetőek fel az oldalunkon!

Fekvőtámasz mester

Végezzen összesen 50 fekvőtámaszt!

Húzózkodás profi

Végezzen összesen 20 Húzózkodást!

Guggolás specialista

Végezzen összesen 100 guggolást!

Újanc futó

Fusson összesen 5 kilométert!

Hasizom király

Végezzen összesen 50 felülést!

Sokoldalú sportoló

Végezzen el 20 fekvőtámaszt, 10 Húzózkodást, 30 guggolást és fusson 1 kilométert!

Haladó sportoló

Érdjen el összesen 100 fekvőtámaszt, 50 Húzózkodást, 150 guggolást és fusson 10 kilométert!

Marathon futó

Fusson összesen 42 kilométert!

Acél törzs

Végezzen összesen 200 felülést!

Extrém sportoló

Végezzen el 200 fekvőtámaszt, 100 húzózkodást, 300 felülést, 300 guggolást és fusson 50 kilométert!

Elért teljesítmények

Ha egy teljesítményt elértél akkor az itt fog megjelenni az alábbiakban...

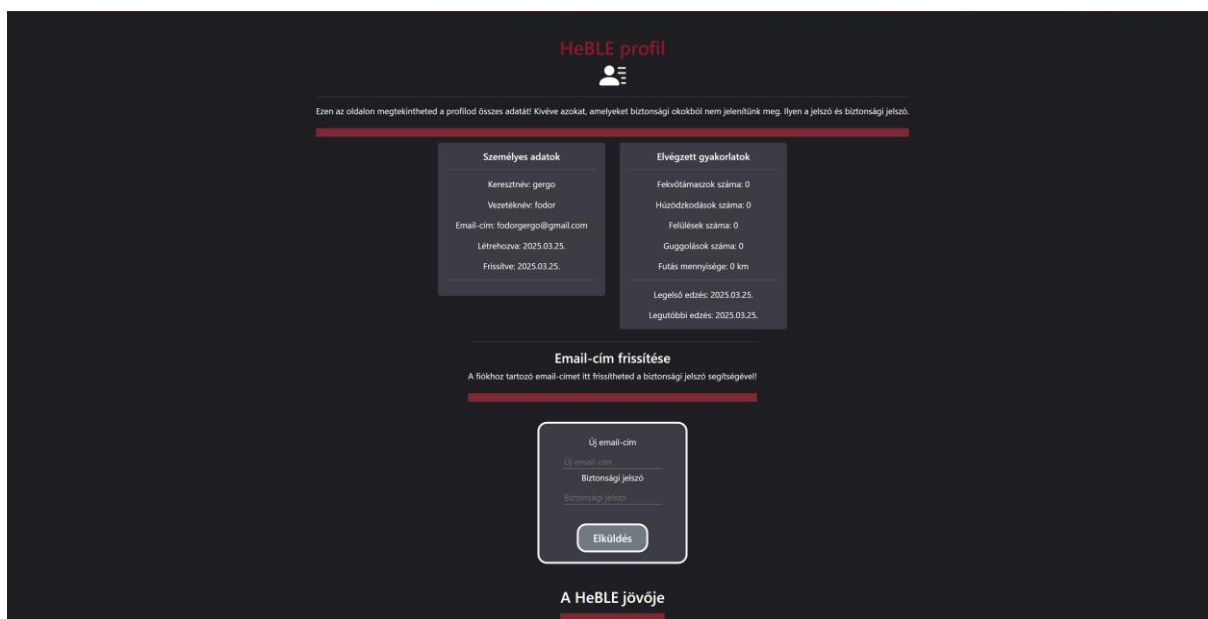
Az elért teljesítmények rész értelemszerűen egy új felhasználó esetén üres, de ahogy a felhasználó teljesíti a különböző teljesítményeket, úgy ezek megjelennek az alábbiakban.



Profil

A Profilnál a felhasználó különböző adatok érhet el. Az egyik ilyen a Személyes adatok, ahol a felhasználó megtekintheti a fiókjához tartozó személyes adatok, amelyeket a regisztrációnál megadott. Ilyen a keresztnév, vezetéknév, az email-cím, hogy mikor lett létrehozva a fiók és, hogy mikor volt utoljára frissítve (például jelszócsere). Emellett közvetlen az Elvégzett gyakorlatok helyezkednek el. Ahol a felhasználó megtekintheti, hogy miből mennyit végzett el eddig összesen. Ez segíti nyomon követni, hogy egy-egy teljesítmény megszerzésében, hogy hol jár. Megtekintheti továbbá az első és a legutóbbi edzés időpontját is a felhasználó. Ezek az adatok folyamatosan frissülnek.

A webalkalmazáson a profilnál elérhető továbbá egy email-cím frissítése funkció, amely segítségével a felhasználó új email-címet rendelhet a fiókjához a biztonsági jelszó segítségével. Az Elküldés gomb mindaddig szürke (ezáltal meg nem nyomható) marad, ameddig az összes mezőt ki nem tölti a felhasználó. A sikertelen és sikeres email-cím frissítésről értesítést kap a felhasználó. Sikeres email-cím frissítés után az oldal frissít egyet és a felhasználó egyből láthatja az új email-címét a személyes adatoknál.



Mindezek alatt a HeBLE jövőjéről olvashatnak a felhasználók. Röviden arról, hogy mit is szeretnénk a jövőben, miben tervezük fejlődni és, hogy mi a végső célunk.

A HeBLE jövője

Mi a tervünk a jövőre nézve?

Szeretnénk, ha a HeBLE nem csak bizalmi alapon működne, hanem a mobilkészülékeken meglévő különböző mozgásos alkalmazásokon keresztül ellenőriznénk, hogy a felhasználó elvégezte-e az adott feladatot. Ilyen alkalmazás az Apple készülékein lévő Egészség alkalmazás, a Samsung készülékein a Samsung Health, de a Google is rendelkezik egy hasonló alkalmazással a Google Fit személyében.

Hogyan tervezünk fejlődni?

Mi a végső célunk?

V. FEJLESZTŐI DOKUMENTÁCIÓ

Telepítés és beindítás menete

Itt a fejlesztői dokumentáció kezdetén a következő lépésekben végig fogjuk vezetni, hogy hogyan kell telepíteni és beindítani a projektünket. Ha bármi hiba vagy probléma lép fel a telepítés vagy beindítás során, akkor nyugodtan keressenek fel minket. Fontos kihangsúlyoznunk a projekt elindításához aktív internetkapcsolatra van szükség. Első lépésként vizsgáljuk meg, hogy mikre lehet szükségünk az elindításhoz.

Először is fejlesztői környezetre, ami legyen a Visual Studio Code (letöltési elérhetősége <https://code.visualstudio.com/>). A letöltést követően menjen végig a telepítési útmutatón.

Ezután szükségünk van a XAMPP-ra (letöltési elérhetősége <https://www.apachefriends.org/>), ami egy ingyenes elérhető nyílt forráskódú szoftvercsomag, amely által elérhető a MySQL/MariaDB adatbázis. A letöltést követően menjen végig a telepítési útmutatón.

Ezek mellett még szükségünk lesz a Node.js-re (letöltési elérhetősége <https://nodejs.org/en>), ami ugye egy nyílt forráskódú szerverfuttatási környezet, verziószám 22.11.0. A letöltést követően menjen végig a telepítési útmutatón.

Emellett nem árt letölteni a Postman-t, ami egy népszerű API-fejlesztői eszköz (letöltési elérhetősége <https://www.postman.com/downloads/>). A letöltést követően menjen végig a telepítési útmutatón.

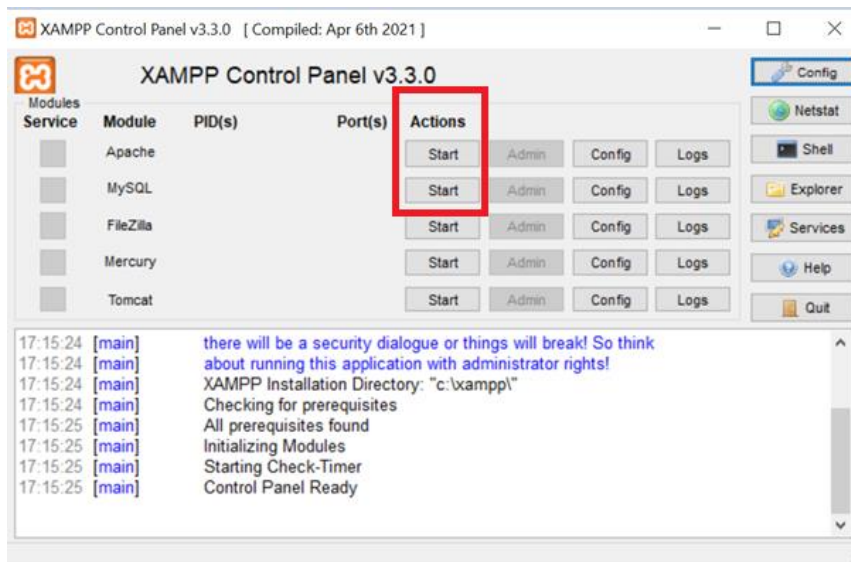
Végül szükségünk lesz még az Angular-ra, ami egy TypeScript alapú nyílt forráskódú webalkalmazás készítő keretrendszer. Ennek telepítését legegyszerűbben a Visual Studio Code-ban tehetjük meg. Ahol nyitunk egy Terminált és oda beillesztjük a következőt: `npm i @angular/cli@19.0.5`

Így tehát most telepítettük sikeresen az összes szükséges elemet a projekt futtatásához. Következő lépésként a következő telepített programot indítsuk el.

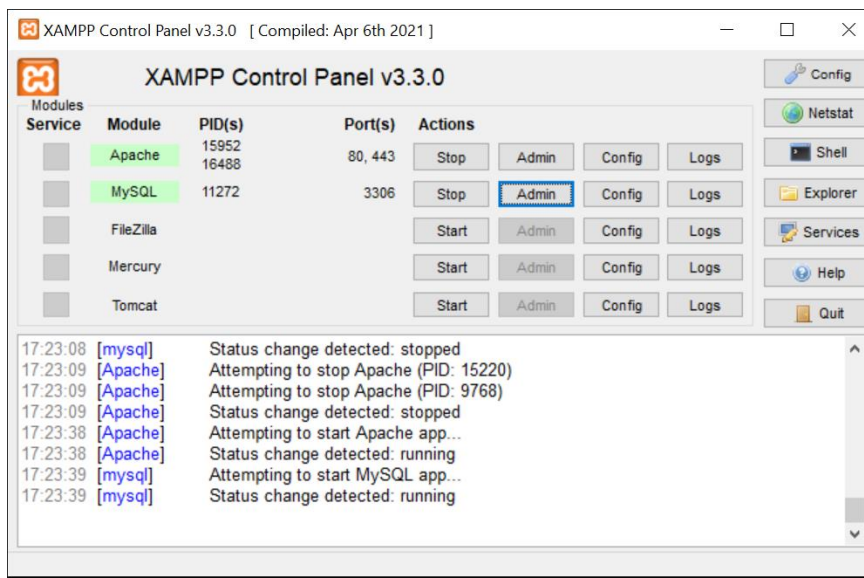


A Postman csak fusson egyelőre a háttérben. A XAMPP esetében lenne pár teendőnk. Először is az Apache és azt követően egyből a MySQL Module esetében az Actions

oszlopban található Start gombokat kell megnyomnunk. Ha minden sikeres, akkor a két Admin gomb elérhetővé vált és a másodikat kell, kiválasztunk és rá kattintanunk (a 2. képen kékkel van épp kiemelve.) Majd Ezeket az alábbi képeken ábrázoltuk a biztonság kedvéért.

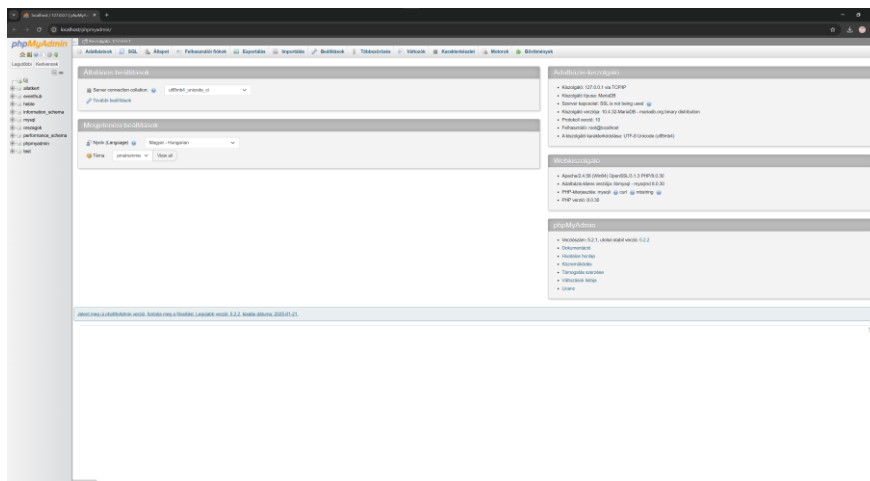


1.



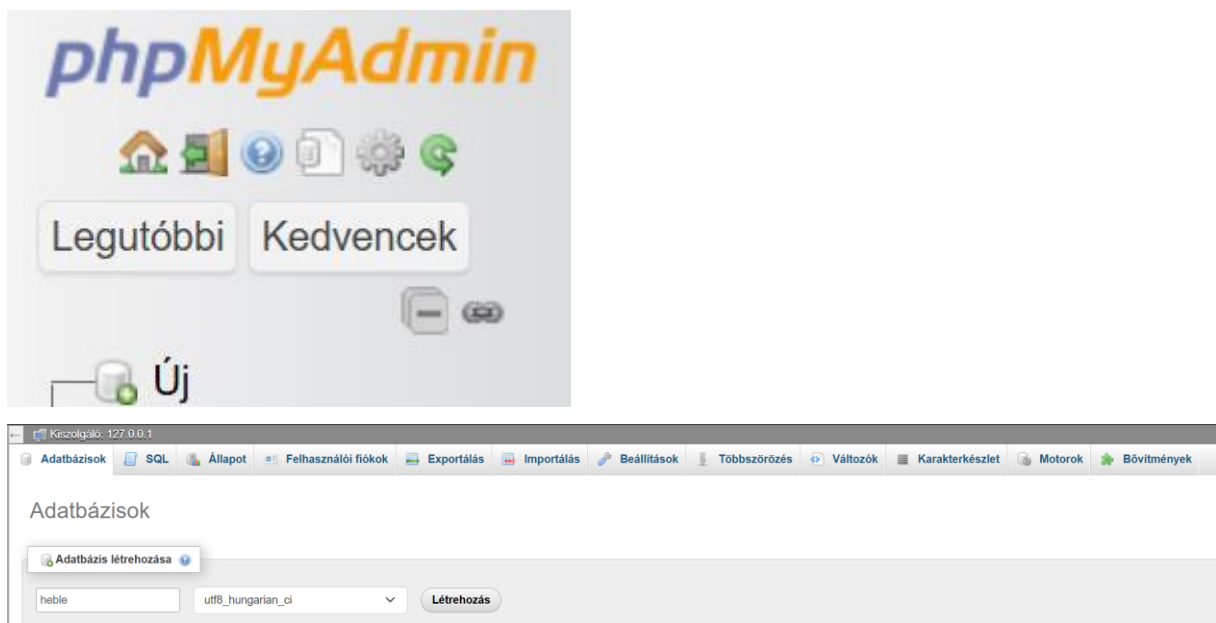
2.

Ezután a következő, a phpMyAdmin felülete nyílik meg egy webböngészőben:



3.

(!) Mivel a projektet valószínűleg most indítja el először így abban az esetben nem rendelkezik a projekthez szükséges alap adatbázissal. Ezt hogyan is hozhatja létre? Nagyon egyszerűen. A bal felső sarokban helyezkedik el az „Új” hozzáadása lehetőség, arra kattintson rá. Ezután át navigálja az Adatbázisok létrehozásához. Itt az adatbázis nevéhez írja be, hogy heble, míg a karakterkódolásnál válassza ki a képen látható módon is a következőt: utf8_hungarian_ci. Majd a létrehozás gomb segítségével létrehozhatja az adatbázist. Ezután, ha minden sikeres, akkor a baloldalon már meg is kellett jelenni az adatbázisunknak.

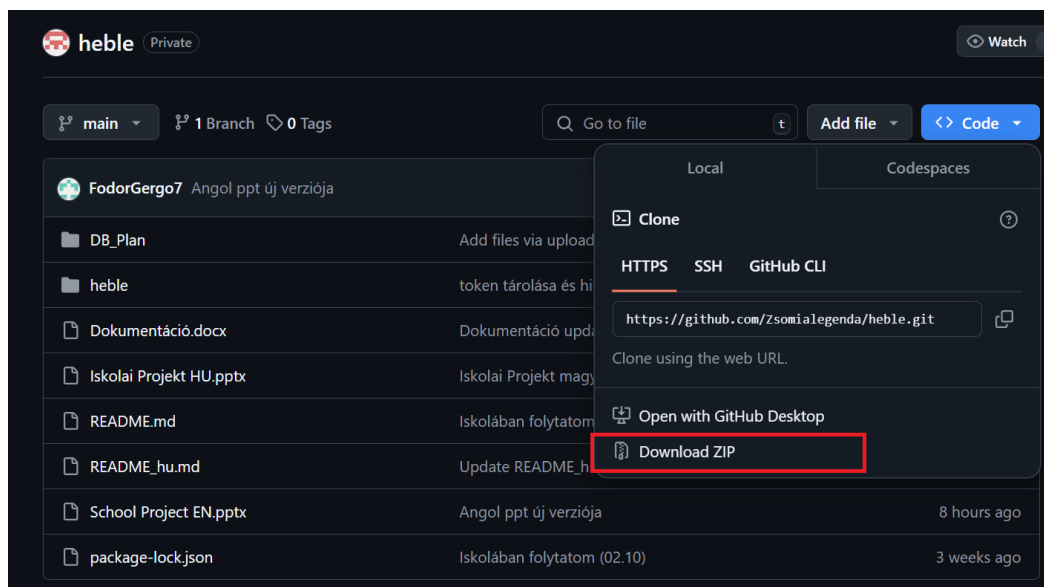


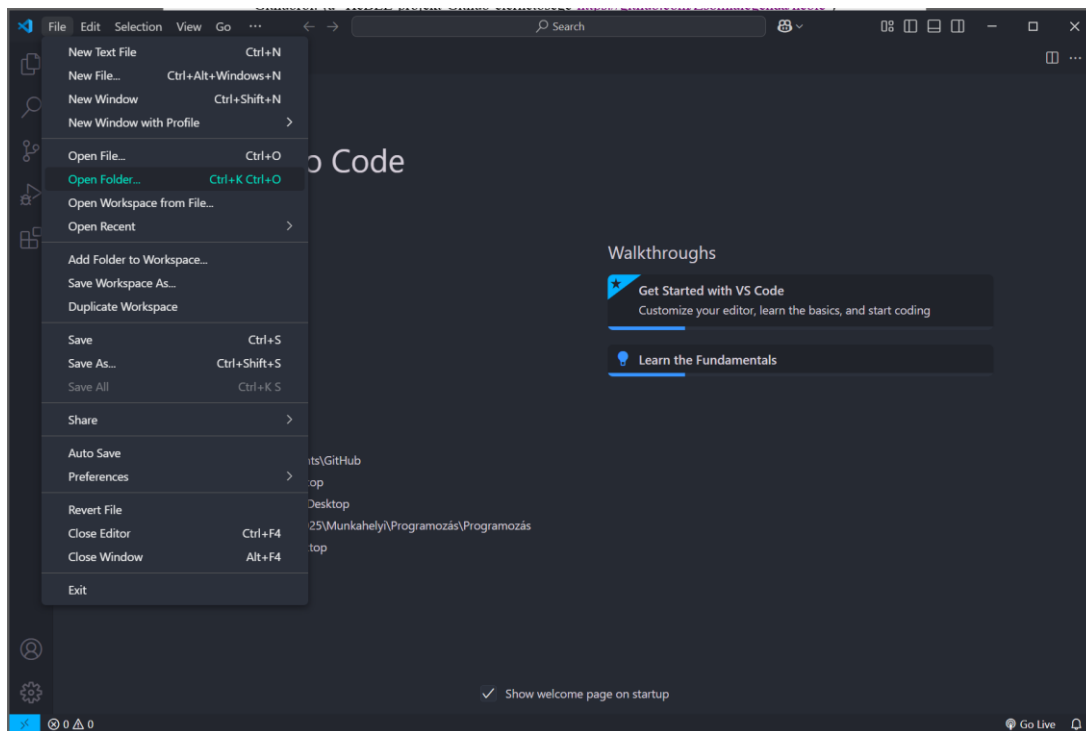
A projekt letöltése és indítása két módon történhet, az egyik manuálisan, a „Download ZIP” segítségével. Azonban a projekt és annak továbbfejlesztése szempontjából

előnyösebb lehet az „Open with GitHub Desktop” lehetőséggel. Ehhez azonban szükségünk lesz a Git-re és a GitHub Desktop-ra, ezeket telepítsük fel először.

Manuális letöltés:

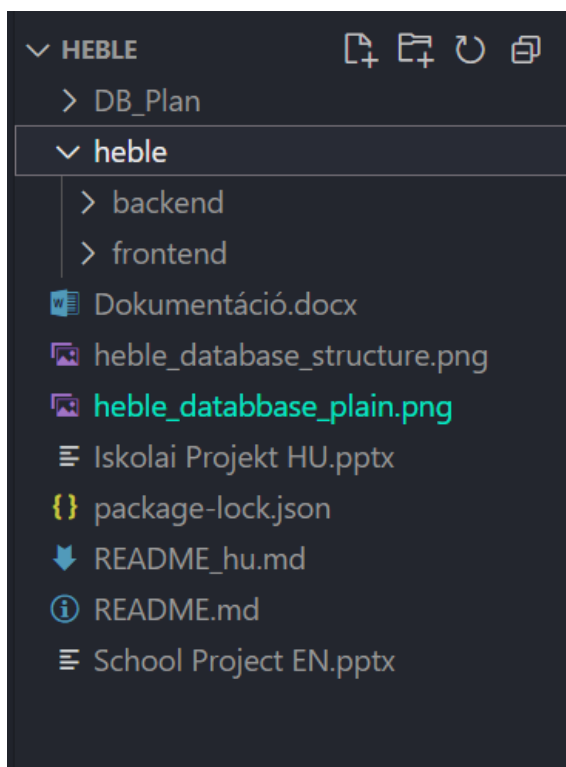
Ha ezekkel sikeresen megvagyunk, akkor most töltsük le az egész becsomagolt projektet Githubról. (a HeBLE projekt Github elérhetősége <https://github.com/Zsomialegend/heble>) Ahogy megérkeztünk az oldalra válasszuk ki a képen látható módon a „Code” fület, majd kattintsunk rá a Download ZIP gombra. A tömörített fájl letöltése után azt csomagoljuk ki és ezután nyissuk meg a Visual Studio Codeban. A második képen látható módon nyithatjuk meg a projektet, azaz a mappát Visual Studio Codeban. Bal fent a „File” gomb, majd az „Open Folder” segítségével, ezután kiválasztjuk a már kicsomagolt HeBLE mappát.





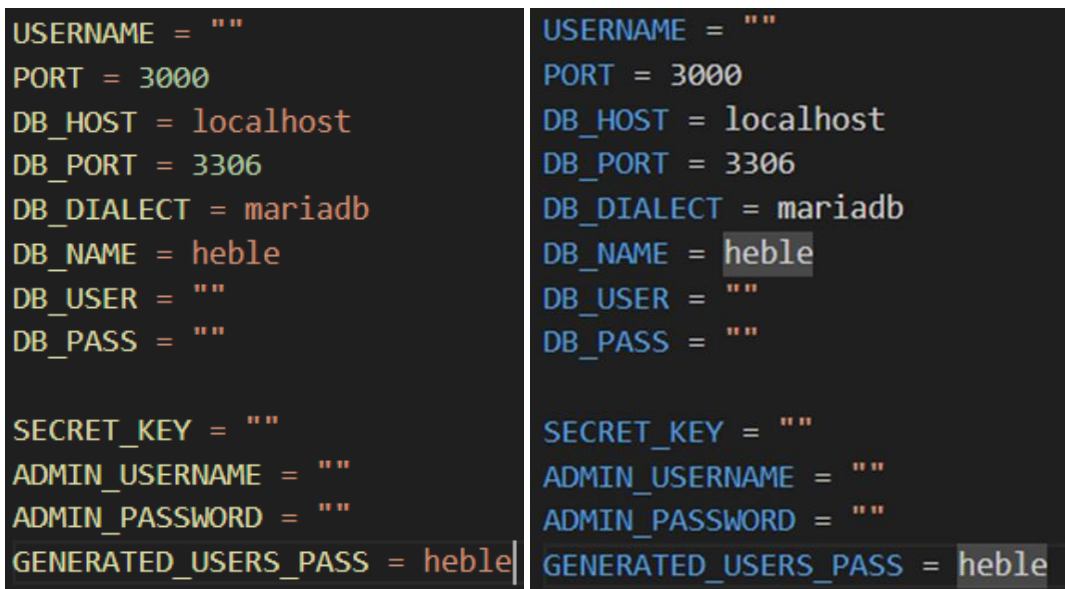
A projekt megnyitása utáni teendők:

A telepítés következő lépésében telepítjük a projekt futtatásához szükséges végső csomagokat, mind a backend és mind a frontend esetében. Végül elindítjuk mindkettőt és már is hozzáférhetünk teljesen a kész projekthez. A megnyitott projektet a baloldalon az Explorer résznél találjuk meg. A következőt láthatjuk, mint projekt szerkezet:



Legelőször is meg kell adnunk bizonyos környezeti változókat a nagyobb biztonság érdekében. Az erre alkalmas fájl a `.env.example` fájl. Ahogy a neve is mutatja ez csak egy minta, ami alapján a fájlt fogjuk megírni. Erre két lehetőségünk van:

1. Legelőször kijelöljük a `.env.example` fájlt és nyomunk egy F2 funkció gombot. Ezután kitöröljük a `'example'` végződést. A fájl tartalmának a színe ilyenkor sárgáról kékre fog változni, jelezve azt, hogy készen áll.
2. Meghagyjuk a példányt és újat csinálunk. Ezt megtehetjük úgy, hogy a fájlt kijelölve **CTRL** + **C** gombkombinációt lenyomjuk. Ezután a forrás mappát (**backend**) kijelöljük. Ezt jelezni fogja a fehér háttér. Majd ezután nyomunk egy **CTRL** + **V** billentyűkombinációt. Így egy új fájl jelenik meg amire nyomnunk kell egy **F2**-t és kitörölni a `'env'` előtti tagokat.



```
USERNAME = ""
PORT = 3000
DB_HOST = localhost
DB_PORT = 3306
DB_DIALECT = mariadb
DB_NAME = heble
DB_USER = ""
DB_PASS = ""

SECRET_KEY = ""
ADMIN_USERNAME = ""
ADMIN_PASSWORD = ""
GENERATED_USERS_PASS = heble
```

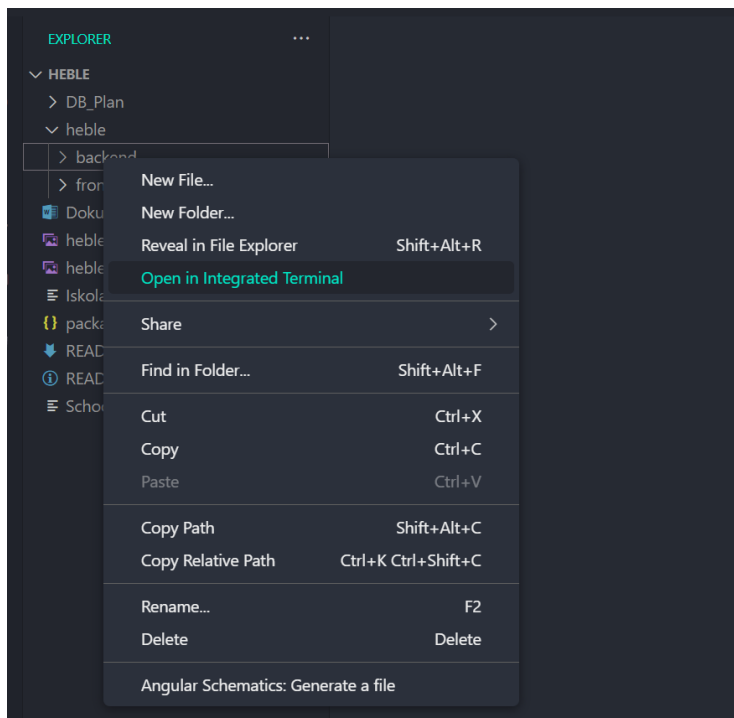
Az első képen a példa fájl látható (sárga szöveg szín), a másodikon a kívánt környezeti változókért felelős fájl (kék szöveg szín).

A fájl tartalmának magyarázata:

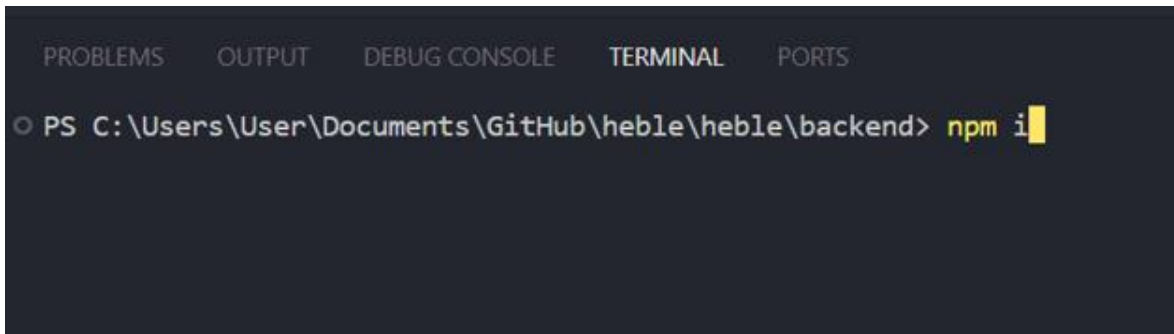
- USERNAME – Felhasználónév
- PORT – Szerver portja.
- DB_HOST – Adatbázis hostja (XAMPP esetén localhost)
- DB_PORT – Adatbázis portja (XAMPP esetén 3306)
- DB_DIALECT – Adatbázis dialektusa (XAMPP esetén MariaDB)
- DB_NAME – Adatbázis neve (a projekthez igazodva heble)

- DB_USER – Adatbázis felhasználója (opcionális)
- DB_PASS – Adatbázis jelszó (opcionális)
- SECRET_KEY – Titkos kulcs, bizonyos titkosításokhoz (pl. token)
- ADMIN_USERNAME – Admin felhasználónév (opcionális, alapértelmezetten admin)
- ADMIN_PASSWORD – Admin jelszó (opcionális, alapértelmezetten admin)
- GENERATED_USERS_PASS – Az előre generált felhasználók jelszava tesztelésekhez (a projekthez igazodva heble)

Tehát először is válasszuk ki a „**backend**” nevű mappát úgy, hogy felé visszük az egeret és jobb kattintással megnyílik majd egy felugró ablak, ahol kiválasztjuk az „**Open in Integrated Terminal**” opciót, ezzel megnyithatjuk a backend mappa terminálját.

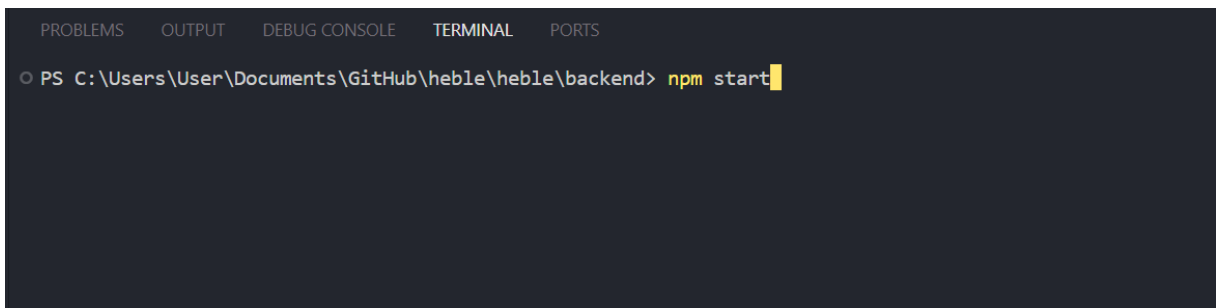


Ahova a következő parancsot kell beillesztenünk: ***npm i***. Ezzel telepítjük a node modulest, amelyben megtalálható az összes függőség, ez szükséges ahhoz, hogy futtatni tudjuk a backendet és elérhessük a projektünket.



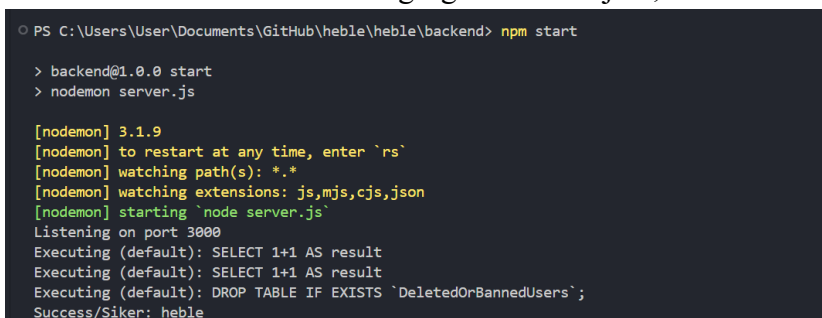
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\User\Documents\GitHub\heble\heble\backend> npm i
```

Ha a parancsot sikeresen lefuttattuk, akkor telepít mindent számunkra szükséges csomagot és könyvtárat a node modules mappába. Ezután futtassuk a backendet, amelyet a következő paranccsal tehetünk meg: ***npm start***



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\User\Documents\GitHub\heble\heble\backend> npm start
```

Ha sikeresen elindult a backend akkor a terminálon láthatjuk, hogy rengeteg adatot töltött fel most az adatbázisba. Ha felgörgetünk a tetejére, akkor a következőket láthatjuk:



```
PS C:\Users\User\Documents\GitHub\heble\heble\backend> npm start

> backend@1.0.0 start
> nodemon server.js

[nodemon] 3.1.9
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
Listening on port 3000
Executing (default): SELECT 1+1 AS result
Executing (default): SELECT 1+1 AS result
Executing (default): DROP TABLE IF EXISTS `DeletedOrBannedUsers`;
Success/Siker: heble
```

Ezután indítsuk el a **frontendet**, a mappát válasszuk ki úgy, hogy felé visszük az egeret és jobb kattintással megnyílik majd egy felugró ablak, ahol kiválasztjuk az „***Open in Integrated Terminal***” opciót, ezzel megnyithatjuk a frontend mappa terminálját.

Ahova a következő parancsot kell beillesztenünk: ***npm i***

Ezzel telepítjük a node modulest, amelyben megtalálható az összes függőség, ez szükséges ahhoz, hogy futtatni tudjuk a backendet és elérhessük a projektünket.

Ha a parancsot sikeresen lefuttattuk, akkor telepít mindent számunkra szükséges csomagot és könyvtárat a node modules mappába. Majd végül indítsa el az Angular alkalmazást ***ng serve --open*** segítségével.

Backend komponens dokumentációja

A projektmunkában a dinamikus backend elkészítése **Sedlák Patrik András** feladatkörébe tartozott

Patrik készítette el az MVC architektúra modell részeit. A modellek sequelize könyvtár segítségével (így JS-t felhasználva) lettek leírva és a heble nevű adatbázisból kérde le az adatokat. A kontrollereket és az azokhoz tartozó útvonalakat a normalizált mappa elnevezések szerint vannak strukturálva.

Bevezetés

Visual Studio Code -- Ehhez a text editorhoz különböző segédcsomagokat is telepítettem, amik segítették a munkát. Ilyenek a Babel JavaScript vagy az **ERD Editor**.

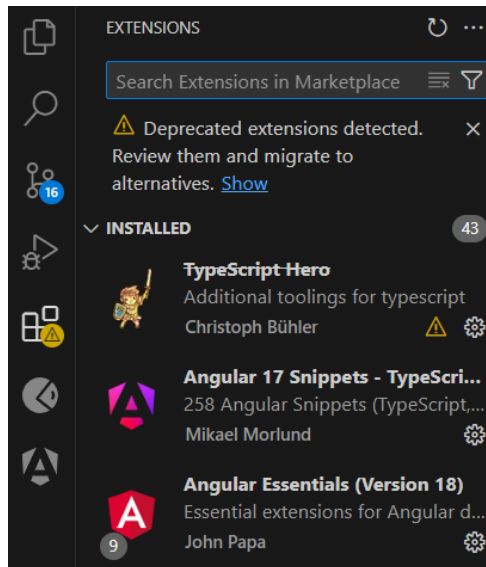
A Babel JavaScript segített a szintaktikai hibák megtalálásába ES201x verzióban.

Az **ERD Editor** volt a leghasznosabb mert azzal elég egyszerűen lehetett vizualizálni az adatbázis kapcsolatokat, és ezzel is készültek a társított **png** formátumú képek a DB_Plan mappában a végleges adatbázisról. Az ehhez tartozó forrásfájl a .../heble/backend mappában található meg heble.vuerd.json néven.

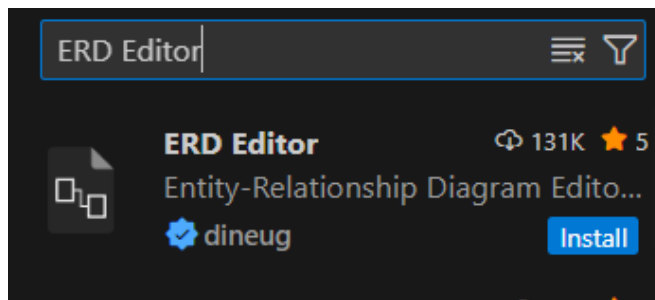
Postman -- Az integrációs tesztelések ebben az API platformban készültek. Ezek a végpont tesztek HeBLE requests.postman_collection nevű fájlban találhatók meg.

ERD Editor telepítése:

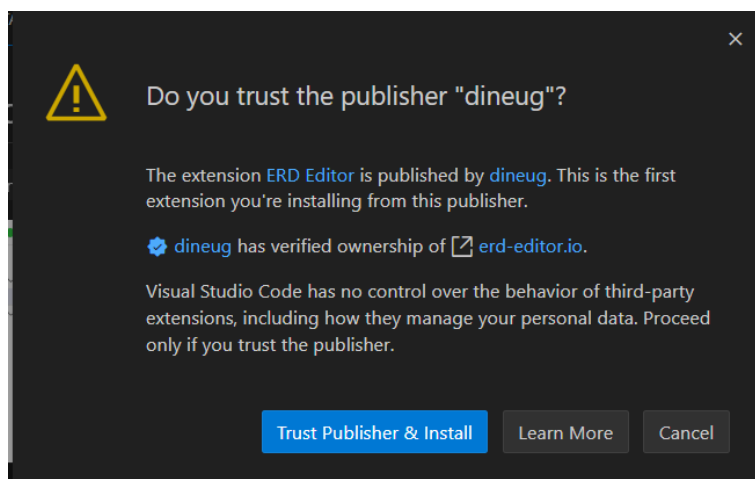
1. Navigálás a *Bővítmények (Extensions)* fülre



2. 'ERD Editor' begépelése



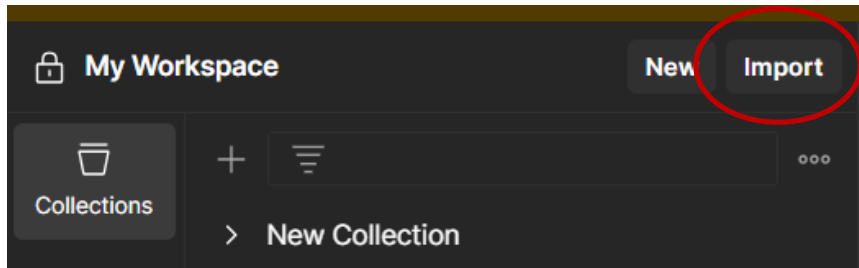
3. 'Install' gomb megnyomása
4. Ha esetlegesen felugrana ez az ablak:



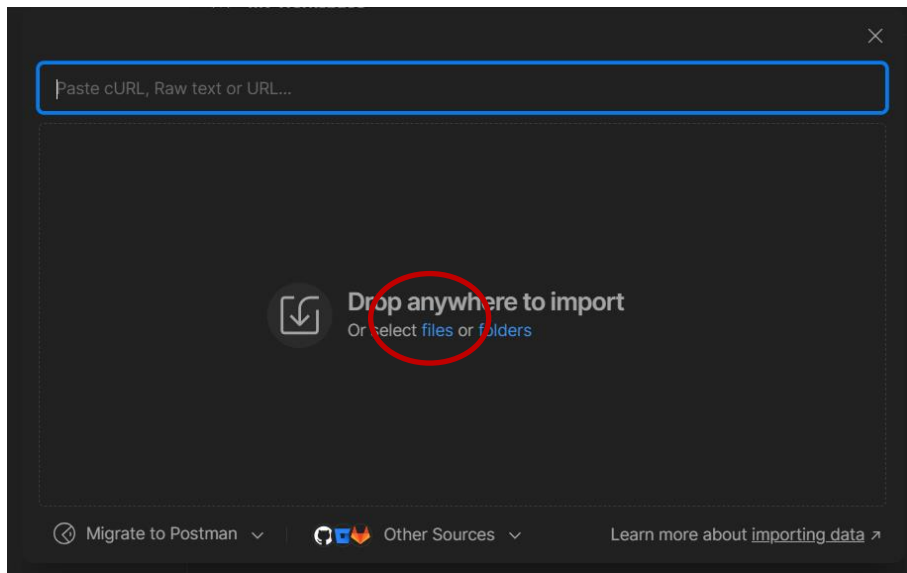
Mivel ez a kiadó megbízható és a VS Code fejlesztői által is hitelesített bővítményről van szó (a pipa a jobb alsó sarokban) így biztosan megnyomható a 'Trust Publisher & Install' gomb.

Postmanbe importálás:

1. Postman megnyitása
2. 'Import' gomb megnyomása



3. 'files' gomb megnyomása



4. A kollekció megkeresése, ami majd a backend forrás mappában lesz
5. Ezután a 'Megnyitás' gombra kattintás.

Adatbázis felépítése

Adatbázisban lévő táblák a következők: User, UserAchievement, UserExperience, Token, Exercise, DeletedOrBannedUsers, Achievement. Ezek a nagyobb biztonság és dinamikusság érdekében vannak legenerálva Sequelizeze-zal.

Táblák magyarázatai:

Az adatok naplózása (*timestamps: true*) lehetővé teszi a létrehozási és módosítási időpontok rögzítését. Ha egy felhasználó törlődik, a kapcsolódó összes többi rekord is törlődik (*onDelete: 'CASCADE'*).

User (Felhasználó):

Ez a tábla a regisztrált felhasználókat tárolja. Minden felhasználónak van egy egyedi azonosítója (*id*), keresztnéve (*firstName*), vezetéknéve (*lastName*), e-mail címe (*email*), jelszava (*password*) és egy biztonsági válasza (*secureAnswer*). A regisztrációkor létrehozott azonosítója a felhasználónak fogja összekötni a felhasználót minden más rekordjával az adatbázisban. A biztonsági válasz a regisztrációkor fel van tüntetve, hogy valami nagyon egyszerű választ is elegendő megadni (például egy háziállat nevet).

Exercise (Gyakorlat):

Ez a tábla egy adott felhasználó által végzett edzéseket tárolja. Minden rekord tartalmaz egy egyedi azonosítót (*id*) és egy felhasználói azonosítót is (*userId*). Minden gyakorlat (*pushUps*, *pullUps*, *sitUps*, *squats*, *running*) alapértelmezetten nulláról indul és számolja a teljesített ismétléseket vagy a futott távolságot (km-ben). Az *onDelete: 'CASCADE'* beállítás biztosítja, hogy ha egy felhasználó törlődik, akkor a hozzá tartozó edzésadatok is eltűnnek.

DeletedOrBannedUser (Törölt vagy Kitiltott Felhasználók):

Ebben a táblában tároljuk azokat a felhasználókat, akik törölve lettek vagy ki lettek tiltva. Minden rekord tartalmaz egy egyedi azonosítót (*id*) és egy felhasználói azonosítót is (*userId*). A *reason* mezőben megadjuk, hogy a felhasználó törölve (*deleted*) vagy kitiltva (*banned*) lett. A *deletedAt* mező naplózza az esemény időpontját.

Achievement (Teljesítmény):

Ez a tábla az elérhető teljesítményeket és kitüntetésekét írja le és tárolja. Minden teljesítmény egyedi azonosítóval (*id*), névvel (*name*) és leírással (*description*) rendelkezik. Meghatározható, hogy egy adott kitüntetés eléréséhez hány *pushUps*, *pullUps*, *sitUps*, *squats* vagy *running* szükséges.

Token:

A bejelentkezett felhasználókhöz tartozó aktív bejelentkezési tokeneket kezeli. A *token* mező tárolja a generált **token** értékét, míg a *loginAt* a bejelentkezési időpontot, és az *expiresAt* a **token** lejáratát rögzíti.

UserAchievement (Felhasználói Teljesítmény):

Ez a tábla tárolja, hogy melyik felhasználó milyen teljesítményt ért el. Minden rekord tartalmaz egy egyedi azonosítót (*id*) és egy felhasználói azonosítót is (*userId*). Az *achievementId* az adott teljesítményre hivatkozik. Az *earnedAt* mező az elérés időpontját rögzíti.

UserExperience (Felhasználói Tapasztalatipont):

Ez a tábla a felhasználók szintjét és tapasztalati pontjait (*xp*) kezeli. A *level* tárolja a felhasználó aktuális szintjét, míg az *xp* mutatja az összegyűjtött tapasztalati pontokat. A *xpToNextLevel* határozza meg, hogy mennyi XP szükséges a következő szintre lépéshez.

Táblák kapcsolatai:

1. Felhasználó és Gyakorlat:

- Egy felhasználóhoz egy gyakorlati tábla tartozhat.
- A Gyakorlat táblában a *userId* idegen kulcs köti össze a Felhasználó táblával.
- Ha egy felhasználó törlésre kerül, a hozzá tartozó gyakorlat rekord is törlődik.

2. Felhasználó és Token:

- Egy felhasználóhoz egy *token* tartozhat.
- A **Token** táblában a *userId* idegen kulcs köti össze a Felhasználó táblával.
- Ha egy felhasználó törlésre kerül, a hozzá tartozó token rekord is törlődik.

3. Felhasználó és Felhasználói Teljesítmény:

- Egy felhasználó több teljesítményt is elérhet, és egy teljesítményt több felhasználó is megszerezhet.
- A kapcsolatot a UserAchievement köztes tábla kezeli, ahol a userId a Felhasználóra, az achievementId pedig a Teljesítményre hivatkozik.
- Ha egy felhasználó törlésre kerül, a hozzá tartozó teljesítmény rekord is törlődik.

4 .Felhasználó és Felhasználói Tapasztalat:

- Egy felhasználóhoz egyetlen tapasztalat rekord tartozhat.
- A **UserExperience** tábla a userId idegen kulccsal kapcsolódik a Felhasználóhoz.
- Ha egy felhasználó törlésre kerül, a hozzá tartozó tapasztalat rekord is törlődik.

5. Teljesítmény és Felhasználói Teljesítmény:

- Egy teljesítményhez több felhasználói teljesítmény rekord tartozhat.
- A **UserAchievement** tábla a achievementId idegen kulccsal kapcsolódik a Teljesítményhez.
- Ha egy felhasználó törlésre kerül, a hozzá tartozó felhasználói teljesítmény rekord is törlődik.

6. Felhasználó és Felhasználói Teljesítmény:

- Egy felhasználóhoz több felhasználói teljesítmény rekord tartozhat.
- A **UserAchievement** tábla a userId idegen kulccsal kapcsolódik a Felhasználóhoz.
- Ha egy felhasználó törlésre kerül, a hozzá tartozó tapasztalat rekord is törlődik.

Kontrollerek

Minden modellhez tartozik egy-egy kontroller, amiben helyenként hasonló függvények operálnak. Mit értek ez alatt?

Minden egyes kontroller fájl `\${modelName}Controller.js` névre hallgat, és mindegyik kontroller első kettő függvénye hasonló funkciókat lát el.

```
▼ controllers
  JS achivementController.js
  JS exerciseController.js
  JS experienceController.js
  JS leaderboardController.js
  JS tokenController.js
  JS userAchivementController.js
  JS userController.js
```

Mindegyik függvény egy rövid *Doxygen Documentation* nevű rövid dokumentációval kezdődik. Az ehhez használt bővítmény a **Doxygen Documentation Generator** névre hallgat. Erre példa:

```
/** signupUser -- felhasználó regisztrálása
 *
 * @param {*} req Az üzenetbe kerül a név, e-mail és jelszó
 * @param {*} res Válaszként vissza küldi hogy a felhasználó létre lett hozva - 201
 * @returns Hibákat küld vissza ha:
 *     1. nincs minden mező kitöltve - 400
 *     2. ki lett tiltva/törölve lett a fiók - 403
 *     3. az e-mail használatban van - 409
 *     4. szerver probléma - 500
 */
```

Az első függvény mindig egy olyan függvény, ami minden adatot lekérdez a modellhez tartozóan (debugolási és fejlesztési célokból néha egyszerre több modell is csatolva van például *getAllUsers()*).

A második függvény pedig mindig egyetlen, a modellhez tartozó rekordot kérdez le. Ezt jellemzően a táblán belüli **azonosító** alapján teszi, vagy ha a felhasználóhoz tartozó rekordról van szó, akkor a felhasználói azonosító alapján történik a lekérdezés (debugolási és fejlesztési célokból néha egyszerre több modell is csatolva van például *getUserByID()*).

Felhasználói Kontroller

Ebbe a kontrollerbe tartoznak a következő függvények:

- **getAllUsers** – Visszaadja az összes felhasználó összes adatát. Továbbfejlesztői lehetőség e függvény implementálása az admin felületen.
- **countUsers** – Visszaadja a felhasználók számát. Az admin felületen van használva.
- **getUserByID** – Visszaad egy felhasználó minden adatát ID alapján. Továbbfejlesztői lehetőség e függvény implementálása az admin felületen.
- **getUserByEmail** – Visszaad egy felhasználó minden adatát e-mail alapján. Az admin felületen van használva.
- **signupUser** – Regisztráció.
- **loginUser** – Bejelentkezés.
- **logoutUser** – Kijelentkezés.
- **updateAccount** – Fiók frissítés.
- **verifySecureAnswer** – A regisztrációkor megadott biztonsági válasz ellenőrzése. Azért szükséges egy kontroller erre, mivel szükséges egy olyan végpont, ahol sikeres eredmény esetén tovább lehet jutni egy másik végpontra. Ezzel a szétszedett végpont módszerrel az adat szivárgás esetlegesen megelőzhető. Továbbfejlesztésként akár egy **token is generálódhat**, ami aztán egy ideiglenes jelszó használatát engedné, amíg az új jelszó elkészül.
- **resetPassword** – Elfelejtett jelszó visszaállítása. Ez a függvény következik a *verifySecureAnswer* után. Itt a felhasználó meg tudja változtatni az elfelejtett jelszót az e-mail és biztonsági válasz segítségével.
- **deleteUser** – Felhasználó törlés. Erre az **admin képes jelenleg** az sajátos jelszavával és az előre legenerált biztonsági válasszal (*'heble'*). Továbbfejlesztési lehetőség az, ha a felhasználó is tudja törölni magát.

Token kontroller

Ebbe a kontrollerbe tartoznak a következő függvények:

- **fetchAllTokens** – Vissza adja az összes token.
- **fetchTokenById** – Vissza ad egy token az azonosítója alapján.
- **fetchTokenByUserId** – Vissza adja a felhasználóhoz tartozó token.
- **countToken** – Vissza adja az aktív tokenek számát, így az aktív felhasználók számát. Az admin felületen van használva.

Gyakorlat Kontroller

Ebbe a kontrollerbe tartoznak a következő függvények:

- **getAllExercises** – Visszaad minden számontartott gyakorlati rekordot.
- **getExercise** - Vissza adja az összesített számát egy gyakorlatnak.
- **getUserExercises** – Visszaadja egy felhasználó gyakorlati rekordját.
- **logExercise** – Ezzel a függvénnyel történik a gyakorlatok feltöltése és a tapasztalati pontok kiszámítása. Az egyes gyakorlatokhoz különböző, de előre meghatározott tapasztalati jutalmak tartoznak. Itt történik meg a **teljesítmények** ellenőrzése is, és a válasz üzenet tartalmazza a közben elért **teljesítményt**.
- **statsExercises** – Visszaadja az egyes gyakorlati mezők összegét statisztikai célokra. Az admin felületen van használva.

Tapasztalat Kontroller

Ebbe a kontrollerbe tartoznak a következő függvények:

- **getAllExperiences** – Minden felhasználóhoz tartozó tapasztalati rekord lekérdezése.
- **getUserExperience** – Egy felhasználóhoz tartozó tapasztalati rekordot ad vissza. A felhasználói felületen van használva.

Achievement Kontroller

Ebbe a kontrollerbe tartoznak a következő függvények:

- **getAllAchievements** – Az összes teljesítmény rekordjának lekérdezése.
- **getAchievementById** – Egy teljesítmény rekordjának lekérdezése azonosító alapján.
- **addAchievement** – Egy achievement hozzáadása a szükséges paraméterekkel. Az admin felületen van használva.
- **updateAchievement** – Egy achievement frissítése az azonosítója, alapján a szükséges paraméterek megadásával. Az admin felületen van használva.

Felhasználói Achievement Kontroller

Ebbe a kontrollerbe tartoznak a következő függvények:

- **getAllUserAchievements** – Minden felhasználó által elért összes teljesítményt adja vissza a következő adatokkal: felhasználó:{azonosító, vezetéknév, keresztnév, e-mail}, teljesítmény:{azonosító, név, leírás}.
- **getUserAchievements** - Egy felhasználó által elért összes teljesítményt adja vissza.
- **getAchievementStats** – Visszaadja, hányan csinálták már meg az egyes achievementeket statisztikai célokra. Az admin felületen van használva.

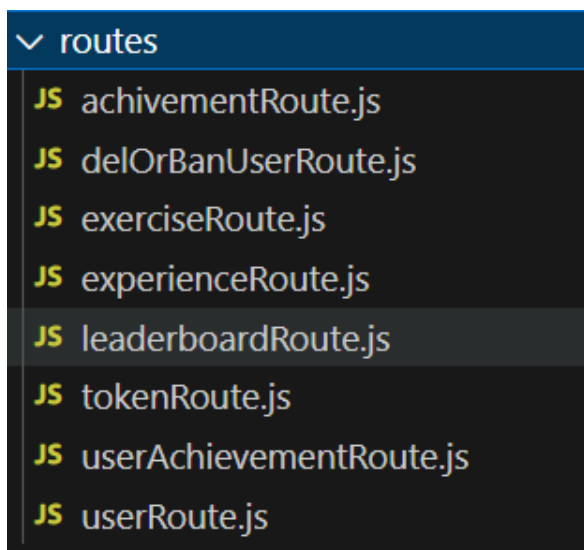
Ranglista Kontroller

Mindegyik az admin felületen van használva.

Ebbe a kontrollerbe tartoznak a következő függvények:

- **getLeaderboardByXP** – XP alapján visszaad egy ranglistát. Ezen a ranglistán látni lehet ki a legváltozatosabb és éppen legszorgalmasabb.
- **getLeaderboardByAchievements** – Achievement számok alapján visszaad egy ranglistát. Ezen a ranglistán látni lehet mennyire motiváltak az emberek teljesítmények elérésében. Tovább fejlesztési lehetőség a kész achievementek neveinek listázása.
- **getLeaderboardByExercise** – Minden eltárolt gyakorlat mezőhöz tartozik egy ranglista. Itt meg lehet nézni az esetleges egyoldalúságokat.

Végpontok dokumentációja



Végpont URL	HTTP Módszer	Leírás	Állapotkód / Válasz
/users - Szülő útvonal			
/	GET	Az összes felhasználó adatainak lekérése jelszó nélkül.	200 OK - Összes felhasználó adat lekérve 500 Internal Server Error - Sikertelen lekérdezés
/ {id}	GET	Egy adott felhasználó adatainak lekérése ID alapján.	200 OK - Felhasználó megtalálva 404 Not Found - Felhasználó nem létezik 500 Internal Server Error - Sikertelen lekérdezés
/email	POST	Felhasználó keresése email alapján.	200 OK - Felhasználó megtalálva 404 Not Found - Felhasználó nem létezik 500 Internal Server

			Error - Sikertelen lekérdezés
/count	GET	A felhasználók számának lekérése.	200 OK - Felhasználók száma 404 Not Found - Nincsenek felhasználók 500 Internal Server Error - Sikertelen lekérdezés
/signup	POST	Új felhasználó regisztrációja.	201 Created - Sikeres regisztráció 400 Bad Request - Érvénytelen adatok 403 Forbidden - Fiók tiltott/törölt 409 Conflict - Email már használatban 500 Internal Server Error - Sikertelen regisztráció
/login	POST	Felhasználó bejelentkezése.	200 OK - Sikeres bejelentkezés 400 Bad Request - Érvénytelen adatok 401 Unauthorized - Hibás email 401 Unauthorized - Hibás jelszó 500 Internal Server Error - Sikertelen bejelentkezés
/logout	POST	Felhasználó kijelentkezése.	200 OK - Sikeres kijelentkezés 400 Bad Request - Érvénytelen adatok 500 Internal Server Error - Sikertelen kijelentkezés
/forgot	PUT	Biztonsági jelszó	200 OK - Sikeres

		ellenőrzése elfelejtett jelszó esetén.	ellenőrzés 400 Bad Request - Hiányzó email/biztonsági jelszó 401 Unauthorized - Helytelen biztonsági jelszó 404 Not Found - A felhasználó nem található 500 Internal Server Error - Sikertelen ellenőrzés
/forgot/reset	PUT	Jelszó visszaállítás.	200 OK - Sikeres visszaállítás 400 Bad Request - Hiányzik az email/új jelszó 404 Not Found - A felhasználó nem található 500 Internal Server Error - Sikertelen visszaállítás
/update	PATCH	Felhasználó adatainak frissítése.	200 OK - Sikeres frissítés 401 Unauthorized - Nincs token 404 Not Found - Nem található felhasználó 400 Bad Request - Nem jó a biztonsági válasz 400 Bad Request - Nincs e-mail megadva 500 Internal Server Error - Sikertelen frissítés

/delete/:id?	DELETE	Felhasználók törlése (rendes és admin felhasználók).	200 OK - Sikeres törlés 401 Unauthorized - Token szükséges 400 Bad request - Hiányzó adatok 404 Not Found - Felhasználó nem található 401 Unauthorized - Érvénytelen token 500 Internal Server Error - Sikertelen törlés
/delorban - Szülő útvonal			
/	GET	Az összes kitiltott/törölt felhasználó lekérdezése	200 OK - Sikeres lekérdezés 500 Internal Server Error - Sikertelen lekérdezés
/:id	GET	Egy adott kitiltott/törölt felhasználó lekérdezése ID alapján.	200 OK - Sikeres lekérdezés 400 Bad request - Nem megfelelő az ID 404 Not Found - A felhasználó nem található 500 Internal Server Error - Sikertelen lekérdezés
/email	GET	Egy adott kitiltott/törölt felhasználó lekérdezése email alapján.	200 OK - Sikeres lekérdezés 400 Bad request - Az e-mail nincs megadva 404 Not Found - A

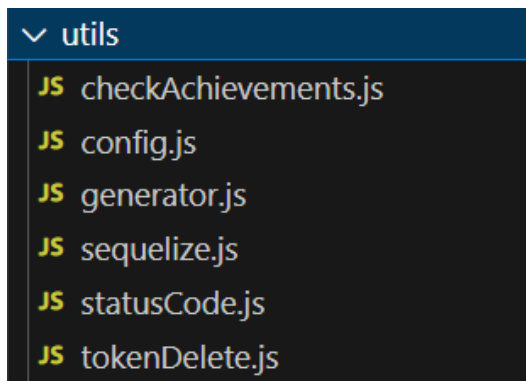
			felhasználó nem található 500 Internal Server - Sikertelen lekérdezés
/achievements - Szülő útvonal			
/	GET	Az összes elért eredmény lekérése.	200 OK - Összes eredmény lekérve 500 Internal Server Error - Sikertelen lekérdezés
/ {id}	GET	Egy adott eredmény lekérése ID alapján.	200 OK - Eredmény megtalálva 400 Bad Request - Érvénytelen adatok 404 Not Found - Eredmény nem létezik
/	POST	Új eredmény hozzáadása.	201 Created - Új eredmény hozzáadva 400 Bad Request – Nincs adat a kérésben 400 Bad Request – Rosszak az adatok 500 Internal Server Error - Sikertelen hozzáadás
/ {id}	PATCH	Létező eredmény frissítése ID alapján.	200 OK - Eredmény frissítve 404 Not Found – A teljesítmény nem létezik 500 Internal Server Error - Sikertelen frissítés

/exercises - Szülő útvonal			
/	GET	Az összes edzés lekérése.	200 OK - Összes edzés lekérve 500 Internal Server Error - Sikertelen lekérdezés
/ {id}	GET	Egy adott felhasználó edzéseinek lekérése ID alapján.	200 OK - Sikeres lekérdezés 400 Bad Request - Érvénytelen adatok 404 Not Found - Felhasználó nem található 500 Internal Server Error - Sikertelen lekérdezés
/stats/sum	GET	Összesített edzésstatisztika lekérése.	200 OK - Sikeres lekérdezés 500 Internal Server Error - Sikertelen lekérdezés
/stats/:type	GET	Egy adott edzéstípus statisztikájának lekérése.	200 OK - Sikeres lekérdezés 400 Bad Request - Érvénytelen edzéstípus 500 Internal Server Error - Sikertelen lekérdezés
/log	POST	Edzésnaplózás és XP szerzés.	200 OK - Sikeres naplózás 401 Unauthorized - Érvénytelen token 400 Bad Request - Érvénytelen ID 403 Forbidden - Csak saját adatok módosíthatók 400 Bad Request -

			Érvénytelen formátum 500 Internal Server Error - Sikertelen naplózás
/leaderboard - Szülő útvonal			
/xp	GET	Ranglista XP alapján.	200 OK - XP alapú ranglista 500 Internal Server Error - Sikertelen lekérdezés
/achievements	GET	Ranglista elért eredmények alapján.	200 OK - Eredmény alapú ranglista 500 Internal Server Error - Sikertelen lekérdezés
/exercise/{type}	GET	Ranglista edzéstípus alapján.	200 OK - Edzéstípus alapú ranglista 500 Internal Server Error - Sikertelen lekérdezés
/tokens - Szülő útvonal			
/	GET	Az összes token lekérése.	200 OK - Összes token lekérve 500 Internal Server Error - Sikertelen lekérdezés
/count	GET	Tokenek számának lekérése.	200 OK - Sikeres lekérdezés 500 Internal Server Error - Sikertelen lekérdezés
/id	GET	Egy adott token lekérése ID alapján.	200 OK - Sikeres lekérdezés 500 Internal Server Error - Sikertelen lekérdezés

/user/{id}	GET	Egy adott felhasználó tokenjeinek lekérése.	200 OK - Sikeres lekérdezés 400 Bad Request - Érvénytelen felhasználó ID 404 Not Found - Token nem található 500 Internal Server Error - Sikertelen lekérdezés
/experiences - Szülő útvonal			
/	GET	Az összes XP pont lekérése.	200 OK - Összes XP lekérve 500 Internal Server Error - Sikertelen lekérdezés
/ {id}	GET	Egy adott felhasználó XP pontjainak lekérése ID alapján.	200 OK - Sikeres lekérdezés 400 Bad Request - Érvénytelen adatok 404 Not Found - Felhasználó nem található 500 Internal Server Error - Sikertelen lekérdezés

Szolgáltatások



Konfiguráció

Itt vannak eltárolva a sikeres adatbázis kapcsolathoz szükséges paraméterek. Az ezekhez szükséges adatokat **környezeti változókként** tároljuk, ami egy esetleges hackertámadás esetén nehezebb hozzáférést fog eredményezni. Ha esetleg nincsen **.env (a környezeti változók fájlja)**, még akkor is a forráskódon belül „helyettesítő” paraméterek, amik a standard paraméterek egy sikeres rákapcsolódáshoz (például az *adatbázis portja* 3306 lesz).

Generátor

Az előre meghatározott Teljesítmények generálása, amiket a felhasználók majd el tudnak érni gyakorlatok feltöltésével. Ha esetleg nem lenne semennyi alany a **faker** könyvtár segítségével random generált rekordokat tölt fel a **Felhasználói** és **Gyakorlati** táblákba. Miközben a generálás történik, a gyakorlatok ellenőrzésre kerülnek és a **Felhasználói Achievementek** táblába is készülnek rekordok. Továbbá itt történik az **admin felhasználó** generálása is. Amint vége a generálásnak kapunk egy üzenetet, ha sikeres.

Sequelize

A konfigurációba írt paraméterek alapján történik a **sequelize** példányosítása, és ezzel együtt az adatbázis kapcsolat is. Miután az autentikáció megtörténik egy üzenetet kapunk a sikeres kapcsolat után. Ha esetleg nem történne kapcsolódás, a hiba üzenet a *konzolon* fog megjelenni. A leggyakoribb hibaüzenet közé tartozik az, ha a szükséges jogok nincsenek meg.

Ilyenkor ezt a SQL mondatot kell futtatni a MySQL admin felületén:

```
GRANT ALL PRIVILEGES ON `<database>`.* TO '<username>'@'<host>'  
IDENTIFIED BY '<password>';  
FLUSH PRIVILEGES;
```

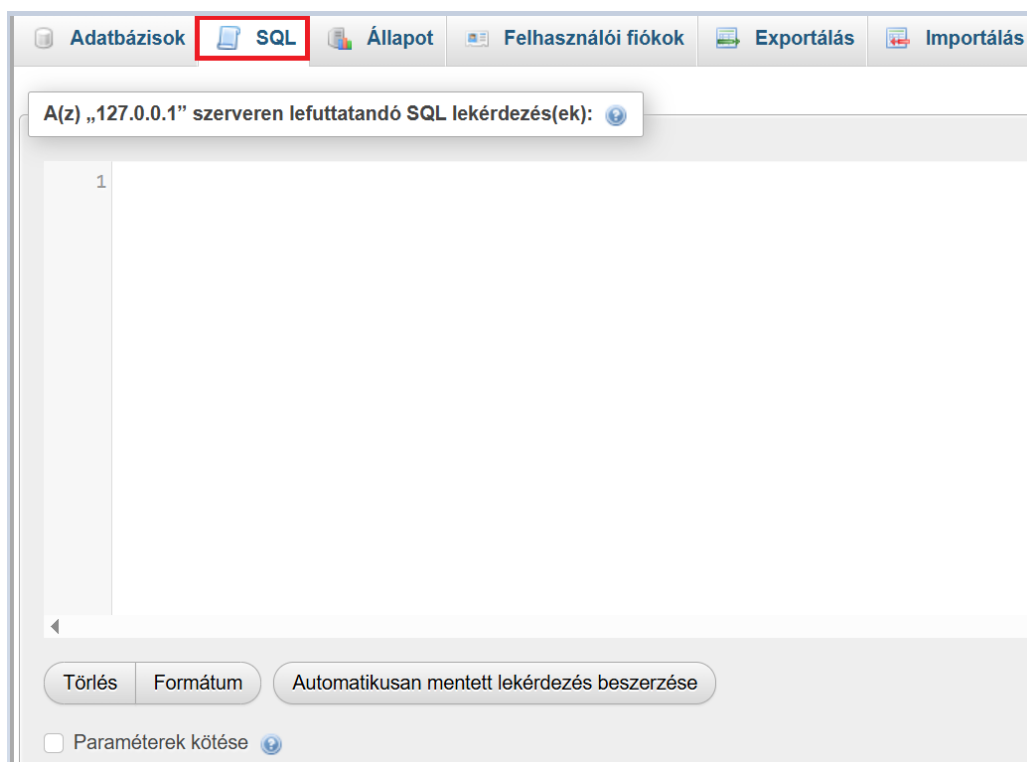
database – a megadott adatbázis név

username – a megadott adatbázis felhasználója

host – a megadott hoszt, aki elérést biztosít az adatbázishoz

password – a megadott jelszó az adatbázishoz

A fent említettem SQL mondatot az itt található admin felület CLI bemásolva és a megfelelő (.env-be írt adatokkal) paraméterekkel lefuttatva érvényes lesz az adatbázishoz való hozzáférés.



Státusz kód

Minden a programban használt **STÁTUSZ KÓD** és az azokhoz tartozó **STÁTUSZ ÜZENET** itt van megírva (a 200-as kivételével). Ezek egyszerű modulok, amik a megfelelő hiba esetén meghívásra kerülnek.

Előnye ennek a szolgáltatásnak az, hogy olvashatóbbá tesz minden kontrollert és azokon belül minden hibakezelést mivel ok/indokot is lehet írni. Hátránya viszont az, hogy mivel egy függvényről van szó, így memória lesz lefoglalva, ami pedig potenciálisan lassíthatja a futási sebességet.

```
const handleError = (status, message, üzenet) => {
  return (error, req, res, next, reason) => {
    if (error) {
      console.error(`Error (${status}):`, error);
      console.error(`Hiba (${status}):`, error);
    } else {
      console.error(`Unknown error occurred (Status: ${status}).`);
      console.error(`Váratlan hiba történt (Status: ${status}).`);
    }

    if (!reason || reason.length < 2) {
      res.status(status).json({
        status,
        message: message,
        üzenet: üzenet,
      });
    }

    res.status(status).json({
      status,
      message: message,
      reason: reason[0],
      üzenet: üzenet,
      indok: reason[1],
    });
  };
};
```

Token törlés

```
const checkAndDeleteExpiredTokens = async () => {
  try {
    const now = new Date();
    const result = await Token.destroy({
      where: {
        expiresAt: {
          [Sequelize.Op.lt]: now,
        },
      },
    });

    console.log(
      `${result} expired tokens deleted. (${result} lejárt token törölve.)`
    );
  } catch (error) {
    console.log("Error deleting expired tokens:", error);
  }
};
```

Ebben a modulban egyetlen függvény található, ami átvizsgálja a **Token** táblát és a lejárt rekordokat elpusztítja. Ez a **szerver-ben** történik meghívásra. Jelenleg 1 órára van állítva, ami azt jelenti, hogy mindig 1 óránként meghívásra kerül egy **setInterval()** nevű függvény, ami futtatja a modult. A bejelentkezési token 8 órára van állítva, mert igazodik az *edző* munkaidejéhez.

Achievement ellenőrzés

Az ebben lévő modulok minden gyakorlat feltöltésnél végbe mennek. Legelőször ellenőrzésre kerül, hogy meg van-e a megfelelő gyakorlat szám az **achievement** eléréséhez, majd, ha meg van, akkor a felhasználó megkapja az **achievementet**.

Authenticator

Ez a modul egy *middleware* ami minden fontos HTTP kérés és válasz között lefut. Ez a modul ellenőrzi, hogy *token* létezik-e a kérés fejlécében vagy sem. Ha létezik a végpontra továbbítja a kérés törzsét, ha nincs válaszként a megfelelő hibát adja vissza.

Frontend komponens dokumentációja

A projektmunka során a reszponzív frontend elkészítése **Dakos János Bence** és **Fodor Gergő** feladatkörébe tartozott.

Gergő készítette el a statikus megjelenését a webalkalmazásnak és annak tartalmait. Ide érthetjük a navigációs menüt és a lábléceket (azok elemeit: kapcsolat és a rólunk), a nyitóoldalt és az ingyenes tartalmak részt is.

A projekt dinamikus funkciói és azok megjelenései közül pedig a regisztráció és a bejelentkezés, a jelszó helyreállítás, az edzés, a teljesítmények és felhasználói profil. Illetve olyan egyéb dinamikus funkciók, mint az autentikáció és route védelem tartozott a feladatahoz.

Bence készítette el a projekthez szükséges egyedi képeket (logó és banner), illetve a teljes adminisztrátori felületet.

Bevezetés

Az összes frontendhez tartozó fájl értelemszerűen a frontend mappában található meg. A fejlesztői környezet a **Visual Studio Code**, az iskolai tanulmányaink során végig ezt használtuk, így ezt ismertük a legjobban. A keretrendszer, pedig az **Angular** volt, annak is a **19-es verziója**. (v19.0.4)

A projekt során használt frontend technológiák a HTML5, CSS3, TypeScript (v.5.6.3). Emellett használtunk különböző Bootstrap ikonokat és komponenseket, mint a navbar, footer, cards, toasts, alerts, forms. Illetve Visual Studio Code-ba hozzáadott különböző bővítmények (Extensions). Ilyen például a Prettier, amit kódformázáshoz használtunk, hogy a kód átláthatóbb és olvashatóbb legyen. Ezzel megfelelve a clean code egyik alapelveinek.

A futtatáshoz szükségünk lesz egy node modules mappára, amelyben megtalálható az összes függőség, ami a projekt elindításához szükséges. A szükséges csomagokat a Frontend mappa terminálján keresztül (jobb egérgomb a mappára, majd az *Open in Integrated Terminal* opciót válasszuk) az *npm i* parancs segítségével telepíthetjük. Majd végül *ng serve --open* segítségével elindíthatjuk az Angular alkalmazást.

app.routes.ts és app.config.ts

Az **app.routes.ts** fájlban adjuk meg az összes elérhető útvonalat és az azokhoz tartozó komponenseket. Egyes útvonalokhoz authGuard-ot és adminGuard-ot rendelünk, amelyek az útvonalakat védik és csak bizonyos feltételeknek megfelelően érhetik el azokat. Egy-egy új útvonal hozzáadása a következőképpen történik:

```
{ path: 'útvonal', component: komponens neve },
```

Az app.routes.ts pár eleme:

Az alábbi útvonalon érhető el a home komponens, aminek definiálása után azonnal automatikusan importálja is.

```
{ path: 'home', component: HomeComponent },
```

Az **app.config.ts** fájl különböző konfigurációs beállításokat tartalmaz, illetve egyéb szolgáltatásokat állítottunk itt be.

Tartalmazza az útvonalak regisztrálását (provideRouter), a withInMemoryScrolling beállítja azt, hogy navigálás esetén mindig a webalkalmazás tetejére dobja felhasználót. Megtörténik a http kliens biztosítása (provideHttpClient segítségével), végül a http interceptor-ok biztosítása történik meg: a tokenhez kötött kérések végrehajtásához (TokenInterceptorService) és a felhasználói munkament ellenőrzésére (SessionService).

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes, withInMemoryScrolling({ scrollPositionRestoration: "top"})),
    provideHttpClient(withInterceptorsFromDi()),
    {provide: HTTP_INTERCEPTORS, useClass: TokenInterceptorService, multi:true},
    {provide: HTTP_INTERCEPTORS, useClass: SessionService, multi:true}
  ]
};
```

services

A services, azaz szolgáltatások, amelyek különböző adatokat kezelnek. A services mappába kerültek a következő Typescript service fájlok: achievement, admin, exercise,

experience, session, token-interceptor és a user service. Ezek a szolgáltatások felelősek adatok lekéréséért és küldéséért különböző függvényeken keresztül.

Példaként a user service-t fogom részletesebben magyarázni. A baseUrl segítségével definiáltam az API alap útvonalát, majd külön-külön változóknak tároltam az egyes végpontokat. Ilyen végpont a signup, a login, a forgot és a forgot/reset.

```
private baseUrl = 'http://localhost:3000/users/';

signupEnd = 'signup';
loginEnd = 'login';
forgotPasswordEnd = 'forgot';
changePasswordEnd = 'forgot/reset';
```

A konstruktorban injektáltam a HttpClient-et, ami lehetővé teszi az adatok kérését és küldését.

```
constructor(private http: HttpClient) { }
```

A **signup** függvény egy POST kérést küld el a /users/signup végpontra egy új felhasználó regisztrálása céljából. Míg **login** függvény egy POST kérést küld el a /users/login végpontra a felhasználó bejelentkezése céljából.

```
signup(users: any) {
  return this.http.post(this.baseUrl + this.signupEnd, users);
}

login(users: any) {
  return this.http.post(this.baseUrl + this.loginEnd, users);
}
```

A user service-be végbemegy továbbá a token dekódolása, a token lekérdezése, a szerepkör kinyerése a tokenből, a UserID kinyerése a tokenből. Emellett olyan függvények találhatók meg a user service-ben, mint az email-cím frissítésért felelős függvény, a jelszóhelyreállításáért és megváltoztatásáért felelős függvény.

A **token-interceptor** service egy különösen fontos service fájl, ugyanis az interceptor a kérésfejlécbé beállítja a felhasználó tokenjét.

```
export class TokenInterceptorService implements HttpInterceptor{

  constructor(private userService:UserService) { }

  intercept(req: HttpRequest<any>, next: HttpHandler):
  Observable<HttpEvent<any>> {

    let tokenizedRequest = req.clone({
      setHeaders: {
        Authorization: "Bearer " + this.userService.getToken()
      }
    })

    return next.handle(tokenizedRequest);
  }
}
```

interfaces

Az interfacek számára létrehoztunk egy interfaces mappát. Az interfaces mappába kerültek a következők: achievement, add-achievement, exercise, experience. Ezek az interfacek felállítanak egy szerkezetet, hogy milyen adatokat várunk el a backend felől.

```
export interface Experience {
  id: number;
  userId: number;
  level: number;
  xp: number;
  xpToNextLevel: number;
}
```

guards

A guards mappába került az auth guard és az admin guard. Ezek a guard-ok felelnek a route-ok védelméért, az az útvonalvédelemért. Az auth guard függvény ellenőrzi, hogy a felhasználónak van-e jogosultsága át navigálni bizonyos útvonalakra. Ha a felhasználó

nincs bejelentkezve, ezáltal jogosultsága nincs megtekinteni az adott tartalmakat akkor a Bejelentkezéshez navigálja, hiába tudja azokat az útvonalakat. Emellett először injektáljuk a user service-t és a router-t a navigáció miatt.

```
export const authGuard: CanActivateFn = (route, state) => {
  const userService = inject(UserService);
  const router = inject(Router);

  if (userService.loggedIn()) {
    return true;
  } else {
    router.navigate(["/login"]);
    return false;
  }
};
```

Az admin guard függvény ellenőrzi, hogy a felhasználónak van-e jogosultsága, az az a szerepköre admin, ezáltal ha az érték igaz akkor át tud navigálni az adminisztrátori felületre. Ha a felhasználó nem admin, akkor át navigálja a felhasználót az Edzéshez. Emellett először itt is injektáljuk a user service-t és a router-t a navigáció miatt.

```
export const adminGuard: CanActivateFn = (route, state) => {
  const userService = inject(UserService);
  const router = inject(Router);

  if (userService.getRole() === 'admin') {
    return true;
  } else {
    router.navigate(["/coaching"]);
    return false;
  }
};
```

Az előzőekben is említése került, hogy ezek a guard-ok útvonalakhoz vannak rendelve az app.routes-ban a következő módon:

```
{ path: 'coaching', component: CoachingComponent, canActivate: [authGuard] },
```

komponensek

Mivel a webalkalmazásunkhoz rengeteg komponenst hoztunk létre, amelyek között rengeteg statikus komponens található, mivel azok könnyen értelmezhetők, így azokról csak röviden írnék pár mondatban.

A webalkalmazás indításakor a felhasználót a **home komponens** fogadja, az állandó **nav** és **footer komponens** között elhelyezkedve. A navigációs menüsor és a lábléc úgynevezett Bootstrap komponensek segítségével lett megoldva. A nav komponens különböző navigációs lehetőségeit a felhasználó státusza határozza meg. (Bejelentkezett állapotba van vagy sem, mi a szerepkörre)

A footer-ben található két navigációs lehetőség. Az egyik az **about komponens**-re navigál, míg a másik a **contacts komponens**re navigál. Ezek a komponensek statikus tartalmakat jelenítenek meg. A **freecontent komponens** és a további **content komponensek** szintúgy statikus tartalmak.

signup komponens

A signup komponens html fájlja tartalmaz egy űrlapot Bootstrap osztályok segítségével, ami a felhasználó regisztrációjáért felel. Az űrlapon hat adatot kérünk be a felhasználótól: Keresztnév, Vezetéknév, Email-cím, Jelszó, Biztonsági jelszó.

Az űrlap mindaddig el nem küldhető, míg az összes mező ki nincs töltve. A html fájlba ellenőrizzük azt, hogy csak akkor tudja elküldeni a felhasználó az űrlapot, ha a jelszó és a biztonsági jelszó minimális hossza 6-6 karakter. Az email-cím érvényességét a komponens Typescript fájljában ellenőrizzük.

A signup() metódus felel a regisztrációért. Az űrlap kitöltése és elküldése után az adatokat továbbítja a user service-ben található signup() függvénynek, ami HTTP POST kérést küld az API-nak. A subscribe() metódussal kezeljük az érkező választ (szervertől). Ha sikeres a regisztráció, akkor azt Bootstrap Toasts felugró értesítés segítségével megjelenítjük. Majd rövid idő után átnavigálja a felhasználót a login komponenshez. Ha a regisztráció sikertelen akkor különböző hibaüzenetek segítségével jelenítjük meg a hibát.

login komponens

A login komponens ugyanezen a logikán alapul. Egy űrlap segítségével bekérjük a felhasználótól a regisztrált fiókjához tartozó email-címet és jelszót. A login() metódus továbbítja az adatokat, majd a user service-ben található login() függvény HTTP POST kérést küld az API-nak. A subscribe() metódussal kezeljük az érkező választ (szervertől). Sikeres bejelentkezés követően egy Bootstrap Toasts felugró értesítés tájékoztat minket, majd rövidesen átirányít az Edzéshez (/coaching). Emellett felhasználóhoz tartozó tokenet eltároljuk a localStorage-ban, a helyi tárolóban.

Sikertelenség esetén is hibaüzenetek tájékoztatják a felhasználót. Ilyen például, ha az email-cím vagy a jelszó hibás (401, 403). Esetleg valami hiba történt a belépés során (500).

```
login() {
  console.log(this.loginUserData);

  this.userService.login(this.loginUserData).subscribe({
    next: (res: any) => {
      localStorage.setItem('token', res.token);

      this.showToast('SuccessfulLogin');

      setTimeout(() => {
        this.router.navigate(['/coaching']);
      }, 2000);
    },
    error: (err: HttpErrorResponse) => {
      if (err.status === 404) {
        this.showToast('missingdataatLogin');
      } else if (err.status === 401 || err.status === 403) {
        this.showToast('unauthorizedLogin');
      } else if (err.status === 500) {
        this.showToast('erroratLogin');
      }
      console.log(err);
    },
  });
}
```

recover login és change-password komponens

Ha egy felhasználó elfelejtené a jelszavát, akkor annak helyreállítására van lehetőség. A jelszó helyreállítás megkezdéséhez a felhasználótól egy űrlap segítségével bekérjük a regisztrációkor megadott email-címét és biztonsági jelszavát. Az egészet végrehajtjuk submitData() metódus segítségével. A submitData() metódus továbbítja az adatokat a user service-ben található forgotpassword() függvény számára, ami HTTP PUT kérést küld az API-nak. A subscribe() metódussal kezeljük az érkező választ.

Ha az email-cím és a biztonsági jelszó egyik, ezáltal sikeres a helyreállítás megkezdése akkor ezt Bootstrap Toasts segítségével megjelenítjük, majd a localStorage-ban eltároljuk a felhasználó email címét végül rövidesen átnavigáljuk a felhasználót a change-password komponenshez.

Hiba esetén különböző hibaüzenetek jelennek meg. Helytelen/nem létező email-cím esetén: a felhasználó nem található (404). Helytelen biztonsági jelszó esetén: Hibás biztonsági jelszó! (401, 403)

```
submitData() {  
  console.log(this.userData);  
  this.userService.forgotPassword(this.userData).subscribe({  
    next: (res: any) => {  
      this.showToast('SuccessfulRecover');  
      localStorage.setItem('email', this.userData.email);  
      setTimeout(() => {  
        this.router.navigate(['/changePassword']);  
      }, 2000);  
    },  
    error: (err: HttpResponse) => {  
      if (err.status === 404) {  
        this.showToast('userNotFound');  
      } else if (err.status === 401 || err.status === 403) {  
        this.showToast('unauthorizedRecover');  
      }  
      console.log(err);  
    },  
  });  
}
```

A jelszó helyreállítás következő szakaszában a felhasználó megadhatja az új jelszavát. A `submitData()` metódus továbbítja az adatokat a `user service`-ben található `changePassword()` függvény számára, ami HTTP PUT kérést küld az API-nak. A `subscribe()` metódussal kezeljük az érkező választ.

Sikeres új jelszó megadást követően egy Bootstrap Toasts értesíti a felhasználót annak sikerességéről, eközben a `localStorage`-ból töröljük az előzőekben eltárolt a felhasználóhoz tartozó email-címet, majd átirányítja a felhasználót a Bejelentkezéshez (`/login`).

```
submitData() {
  console.log(this.userData);

  this.userService.changePassword(this.userData).subscribe({
    next: (res: any) => {
      this.showToast('SuccessfulPasswordChange');
      localStorage.removeItem('email');

      setTimeout(() => {
        this.router.navigate(['/login']);
      }, 2000);
    },
    error: (err: HttpResponse) => {
      this.showToast('SomethingWentWrong');
    },
  });
}
```

coaching komponens

A komponens útvonala `AuthGuard` által védett, így csak bejelentkezett felhasználók férhetnek hozzá. A komponens betöltésekor az `ngOnInit()` metódus lekérdezi a felhasználó előrehaladását, majd azokat megjelenítjük egy Bootstrap Card-on belül elhelyezve. Megjelenítjük a felhasználó szintjét (level), tapasztalatpontját (xp) és a következő szintig szükséges tapasztalatpontját.

Az `addExercise()` metódus segítségével az opcionálisan kiválasztott gyakorlattípusokból elvégzett gyakorlatokat adhat hozzá a felhasználó. Az `addExercise()` metódus továbbítja az adatokat az `exercise service`-ben található `addExercise()` függvény számára, ami HTTP

POST kérést küld az API-nak. A `subscribe()` módszerrel kezeljük az érkező választ. A felhasználó egy űrlap kitöltése és elküldése után visszajelzést kap Bootstrap Alert-ek formájában, hogy sikeres vagy hiba esetén a sikertelen gyakorlatrögzítésről. A gyakorlat(ok) sikeres hozzáadását követően az oldal frissít (reload-ol) egyet. Így a felhasználó egyből megtekintheti a frissített előrehaladást.

```
addExercise() {
  this.exerciseService.addExercise(this.exercise).subscribe({
    next: (res: any) => {
      this.successMessage = 'Sikeres gyakorlatrögzítés!';
      setTimeout(() => {
        window.location.reload();
      }, 1500);
    },
    error: (err: HttpResponse) => {
      this.errorMessage = 'Sikertelen gyakorlatrögzítés!';

      setTimeout(() => {
        this.errorMessage = '';
      }, 3000);
    },
  });
}
```

achievements komponens

Az achievement komponens-nél a felhasználó megtekintheti a webalkalmazáson elérhető összes teljesítményt, ezeket statikus jelenítjük meg először. A komponens Typescript fájljában a komponens betöltésekor lefutó `ngOnInit()` módszer, ami a felhasználó által már elért teljesítmények megjelenítésért felel. Ha van elért teljesítmény akkor ezeket dinamikus módon megjeleníti egy számozatlan felsorolás segítségével.

```
<ul>
  <li *ngFor="let achievement of achievements">{{achievement.Achievement.name}}
</li>
</ul>
```

userprofile komponens

A komponens betöltésekor lefutó `ngOnInit()` metódus betöltésekor lekérdezi a bejelentkezett felhasználók személyes adatait és elvégzett gyakorlatait külön egy-egy Bootstrap card (kártya) segítségével.

Ezen adatok alatt a felhasználónak lehetősége van a fiókjához tartozó email-cím frissítéséhez. A felhasználó az űrlap egyik mezőbe megadhatja az új email-címet majd a biztonsági jelszó segítségével igazolhatja magát. Az `submitData()` metódus továbbítja az adatokat a user service-ben található `updateEmail()` függvény számára, ami HTTP PATCH kérést küld az API-nak. A `subscribe()` metódussal kezeljük az érkező választ. A sikeres és sikertelen email-cím változtatásról az értesítést Bootstrap alert-ek biztosítják.

A profil résznél megtalálható még egy rövid rész a HeBLE jövőjéről, ezt Bootstrap Accordion segítségével oldottuk meg, mint összezárható elemek.

```
submitData() {  
  console.log(this.userData);  
  
  this.userService.updateEmail({ newEmail: this.userData.newEmail,  
secureAnswer: this.userData.secureAnswer }).subscribe({  
    next: (res: any) => {  
      this.successMessage = 'Sikeres email-cím frissítés!';  
      setTimeout(() => {  
        window.location.reload();  
      }, 1500);  
    },  
    error: (err: HttpResponse) => {  
      this.errorMessage = 'Sikertelen email-cím frissítés!';  
  
      setTimeout(() => {  
        this.errorMessage = '';  
      }, 3000);  
    },  
  });  
}
```

Admin felület

Bevezetés

Ez nem teljesen egy rendszeradminisztrátor, hanem sokkal inkább egy adminisztrátori jogosultságokkal bíró moderátor/operátor. Az itt megtalálható elemek a felhasználókkal, a gyakorlatokkal és a teljesítményekkel kapcsolatosak.

Adminpanel komponens

Ez a szülőkomponens. Az itt megtalálható dinamikus elemek statisztikai dolgok. Itt ranglistákat találhatunk, de erre épül minden további komponens, melyeknek a neve tükrözi a tartalmait.

Ranglisták		
<u>Fekvőtámasz ranglista</u>		
Helyezés	Felhasználónév	Fekvőtámaszok
1	Mabelle Mueller	198
2	Coty Fahey	192
3	Cicero Armstrong	187
4	Rene Quitzon	170
5	Billie Ortiz	159
6	Admin User	152
7	Marcelle Ziemann-Erdman	98
8	Ford Morar	93
9	Tomasa Daniel	56
10	Danny Kub	31

A ranglisták minden gyakorlat típusra, tapasztalati pontra és elért teljesítményekre vonatkoznak. Ezeknek a fontossága megfigyelni az egyoldalúságot és a nehéz munkát, mivel valószínűleg lesznek olyan felhasználók, akik több munkát fognak befektetni egyes dolgokba.

```

ngOnInit() {
  this.loadAchievementChart();
  this.loadExerciseChart();
  this.fetchLeaderboard('pushUpsLeaderBoard', 'exercise', 'pushUps');
  this.fetchLeaderboard('pullUpsLeaderBoard', 'exercise', 'pullUps');
  this.fetchLeaderboard('sitUpsLeaderBoard', 'exercise', 'sitUps');
  this.fetchLeaderboard('squatsLeaderBoard', 'exercise', 'squats');
  this.fetchLeaderboard('runningLeaderBoard', 'exercise', 'running');
  this.fetchLeaderboard('xpLeaderBoard', 'xp');
  this.fetchLeaderboard('userAchievementLeaderBoard', 'achievements');
}

```

A kódrészletben látható **loadAchievementChart** meghívja az achievement grafikont, az alatta lévő **loadExerciseChart** pedig a gyakorlatok grafikont hívja meg. A **fetchLeaderboard** metódus pedig a ranglisták adatait kéri le (*Fekvőtámasz, Húzódkodás, Felülés, Guggolás, Futás, Tapasztalatipont és Elért teljesítmények*).

```

fetchLeaderboard(tableId: string, category: string, type: string = ''): void {
  this.adminService.getLeaderboard(category, type).subscribe({
    next: ({ users }) => {
      if (Array.isArray(users)) {
        this.leaderboards[tableId] = users.map((user) => ({
          ...user,
          username: user.firstName + ' ' + user.lastName,
        }));
      } else {
        console.error(
          'Hiba: A leaderboard API nem tartalmaz megfelelő adatokat:',
          users
        );
        this.leaderboards[tableId] = [];
      }
    },
    error: (error) => {
      console.error(`Hiba a ${tableId} leaderboard betöltésekor:`, error);
      this.leaderboards[tableId] = [];
    },
  });
}

```

Ebben a kódrészletben a **this.adminService.getLeaderboard(category, type)** lekéri a különböző kategóriákhoz tartozó ranglistát. Az adatok egy API-ról érkeznek, mely egy **users** nevű felhasználói listát küld vissza. Ha az adatok sikeresen megérkeznek, akkor az (**Array.isArray(users)**) ellenőrzi, hogy tömbben vannak-e. Ha igen, akkor minden felhasználóhoz hozzáad egy **username** mezőt, majd az így kapott listát elmenti a **this.leaderboards[tableID]** objektumba.

Ha az adatok nem jók, nincs találat vagy valami hiba történt akkor a konzolba ír egy hibaüzenetet, hogy lássuk, mi a probléma, majd elment egy üres listát, hogy az admin felületen ne jelenjen meg hibás adat.

A típus lehet az egyes gyakorlatok (exercise), a tapasztalati pont (xp), illetve a teljesítmény (achievement), míg a kategória elválasztja a gyakorlatokat egymástól.

Admin-achievement komponens

Ez a komponens felel az achievementek (teljesítmények) bizonyos manipulálásáért. Itt lehet új achievementet létrehozni, illetve már meglévő achievementet frissíteni. Ennek a komponensnek a feladata az új akadályok létrehozása és egy videojáték-szerű élmény átadása.

Achievement (Teljesítmények) frissítése	Achievement (Teljesítmények) hozzáadása
Achievement ID: 	Achievement neve:
Achievement neve: 	Achievement leírása:
Achievement leírása: 	Elvárt fekvőtámaszok:
Szükséges fekvőtámaszok: 	Elvárt húzózkodások:
Szükséges húzózkodások: 	Elvárt felülések:
Szükséges felülések: 	Elvárt guggolások:
Szükséges guggolások: 	Elvárt futásmennyiség (km-ben):
Szükséges futás (km-ben): 	
Achievement frissítése	Achievement hozzáadása

Admin-user-management komponens

Itt találhatóak meg a felhasználókra vonatkozó menedzselések. Itt lehet felhasználókra rákeresni e-mail alapján, illetve törölni őket a regisztrációkor generált felhasználói azonosító (ID) alapján. Az edzőknek lesz lehetőségük az adminokkal kommunikálni és megnézni az egyes felhasználóknak az előrehaladását a kereséssel, illetve ha valamilyen csalás történne, akkor ki lehessen őket törölni.

A felhasználó törlése során az email-címén kívül még az adminisztrátori jelszó (a .env fájlba megadott ADMIN_PASSWORD) és a biztonsági jelszó (mivel előre generált

felhasználó ezért a jelszó: „heble”) megadásai kötelezőek. Így biztosítva van a maximális védelem a kényszerű támadások ellen.

Felhasználó Törlése

Felhasználó email-címe:

Adminisztrátori jelszó:

Biztonsági jelszó:

Felhasználó törlése

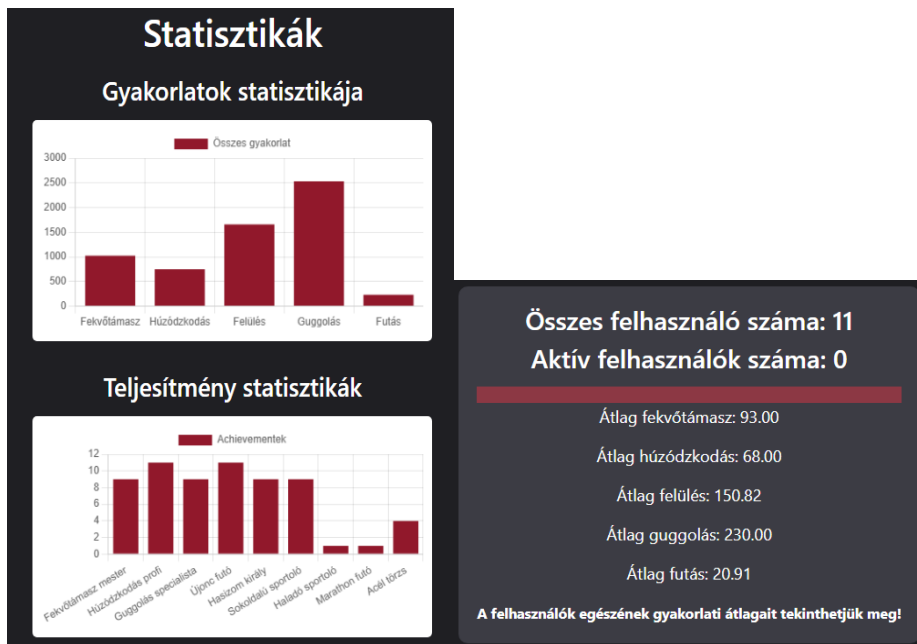
Felhasználó Keresése

Felhasználó email-címe:

Keresés

Admin-statistics komponens

Itt található a felhasználók gyakorlati és elért achievementek által lekérdezett statisztikák. Ide tartozik a két diagram és az összes felhasználóra vonatkozó átlagok az egyes gyakorlatokból.



```

async loadExerciseChart() {
  this.adminService.getExerciseStats().subscribe({
    next: (res: any) => {
      const labels = [
        'Fekvőtámasz',
        'Húzódzkodás',
        'Felülés',
        'Guggolás',
        'Futás',
      ];
      const data = [
        res.totalPushUps || 0,
        res.totalPullUps || 0,
        res.totalSitUps || 0,
        res.totalSquats || 0,
        res.totalRunning || 0,
      ];
    }
  });
}

```

Az ebben a kódrészletben található **loadExerciseChart()** függvény lekéri az elvégzett gyakorlatok statisztikáit és megjeleníti egy oszlopdiagramon. Az **adminService.getExerciseStats()** egy http kérést küld a szervernek, amely visszaküldi az összes gyakorlati adatokat. Ezek után beállítja a diagram címkéit (**const labels = []**), majd beállítja az adatait (**const data = []**). Az API válaszából összegyűjti a gyakorlatok összmennyiségét. Ha egy érték nem definiálható vagy nulla, akkor az alap értéket nullára állítja.

VI. TESZTELÉS

Backend tesztelés

A tesztelés Postman-en keresztül zajlott (ennek importálása feljebb). A végpontok közvetlenül megírásuk után lettek tesztelve. Az egyszerűbb lekérdezések (pl. `getAllUsers()`) egységtesztek, az összetettebb kérések (pl. `logExercise()`) pedig implementációs tesztek, mivel a használatukhoz más funkciókat is használnunk kell (pl. ehhez be kell jelentkezünk mivel a bejelentkezéskor generált token elvárt).

Bár az első 10 felhasználó mindig random generált, viszont a nagyobb adatmennyiség érdekében a forrásmappában létezik egy **'database.sql'** fájl, aminek a tartalmát ha bemásoljuk a XAMPP admin felületén lévő SQL menübe, nagyobb adatállománnyal tesztelhetünk. Az ezekhez tartozó bejelentkezéshez szükséges (és így teszteléshez is) adatok egy **'data.txt'** nevű állományban találhatók meg.

Frontend tesztelés

A frontend tesztelés célja, hogy ellenőrizzük, hogy a funkciók jól működnek-e, minden az elvárt eredmények szerint megy-e végbe. A webalkalmazás komponensei közötti kapcsolat helyesen működik.

Teszteset 1. – Sikeres regisztráció

Ellenőrizzük a sikeres regisztrációt legelőször.

- az összes beviteli mező kitöltése a regisztrációs űrlapon
- helyes adatokkal: jelszó – minimum 6 karakter, email-cím megfelel a feltételeknek
- regisztráció gomb megnyomása
- a rendszer elküldi az adatokat, érvényesíti, majd menti az adatokat adatbázisba
- várt eredmény: a regisztráció sikeres
- üzenet: Sikeres regisztráció!
- átnavigál a Bejelentkezéshez

Hiba felmerülése:

- Ha az email címmel már regisztrált egy felhasználó, akkor a regisztráció sikertelen. Üzenet: Az email cím már foglalt!
- Ha valamilyen hiba merül fel, például nem érhető el a backend. Üzenet: Valami hiba történt a regisztráció során!

Teszteset 2. – Sikertelen jelszóhelyreállítás

A felhasználó regisztráció után be akar jelentkezni, de sikertelenül, mert elfelejtette a jelszavát. Át irányít az „Elfelejtettem a jelszavam!” segítségével a jelszóhelyreállításhoz.

- az összes beviteli mező kitöltése
- email-cím beírása helytelenül, biztonsági jelszó megadása helyesen
- elküldés gomb megnyomása
- várt eredmény: sikertelen
- üzenet: A felhasználó nem található!
- email-cím beírása helyesen, biztonsági jelszó megadása helytelenül
- várt eredmény: sikertelen
- üzenet: Hibás biztonsági jelszó!

Teszteset 3. – Sikeres gyakorlathozzáadás és teljesítmény elérés, elvégzett gyakorlatok

A felhasználó bejelentkezés után sikeresen hozzáad különböző gyakorlatokat. A gyakorlatok között van olyan, amivel teljesítmény ért el a felhasználó, így az megjelenik az elért teljesítményeknél.

- minden gyakorlathoz 10-es szám beírása
- Beküldés gomb megnyomása
- sikeres gyakorlatrögzítés
- frissítés után látható az előrehaladás
- várt eredmény: 1 elért teljesítmény
- tényleges eredmény: 1 elért teljesítmény – Újonc futó
- várt eredmény: Profilnál megtekinthető az eddig elvégzett összes gyakorlat
- tényleges eredmény: minden a rögzített gyakorlatok szerint frissült

Terheléses tesztelés

A webalkalmazás egyszerre több böngészőn futtattuk. Az adatbázisba relatív kis időtartam alatt próbáltunk adatokat felvinni. Az adatfelvitel sikeres volt, így kevés (10 kliens szám alatt) sikeresen működött. Így a terheléses tesztelés is sikeresen zárult.

VII. Továbbfejlesztési lehetőségek

Rengeteg továbbfejlesztési lehetőség rejlik ebben a projektben azonban a sok ötlet megvalósításához sokkal több erőforrásra, azaz energiára és időre van szükség. Így ide azokat a továbbfejlesztési lehetőségeket soroljuk, amiket nem tudtunk megvalósítani, de ezekkel lehetne bővíteni a szoftvert.

Ezek között vannak tényleg egyszerű dolgok, mint például **több ingyenesen olvasható tartalom**. Több tartalom = több időt tölt el felhasználó a webalkalmazásunkon. Emellett még több különböző gyakorlatok, jutalmak és teljesítmények elérése és hozzáadása színesítené ezt a felhozatalt, illetve a felhasználói élményt.

A **véleményeket a felhasználók a webalkalmazáson keresztülírhatnák meg**, nem emailre történő üzeneten keresztül. Ezt úgy lehetne megvalósítani, hogy a regisztrált és jelentkezett felhasználók számára lenne egy kitölthető mező, amelybe megfogalmazhatják a véleményüket és elküldhetik. Ezeket az oldal adminisztrátorai megkapnának és egy rövid elbírálás után (ami akkor sikeres, ha a felhasználó korrektül, helyesírási hibákat keveset vagy egyáltalán nem tartalmazó, kulturált szöveget fogalmazott meg az adminisztrátor megítélése szerint) megjelenítjük a nyitó oldalon.

Az oldalon a felhasználók kiválaszthatnák, hogy milyen **nemhez és korcsoportoz tartoznak**, és a szerint kapnák a feladatokat, így nem csak a mi korosztályunkat, generációkat szólíthatnánk és érhetnénk el, hanem a nálunk fiatalabb korosztályt és a nálunk jóval idősebb korosztályokat is.

Aki még inkább kihívásokra vágya, az lehetőséget kapna **nehézségi fokok kiválasztására**. Így nem csak nagyobb jutalmakat elérve az alkalmazásban, de a felhasználó akár valódi eredménynek is eltitulálhatja.

Az oldal jelenlegi állapota szerint becsület alapon működik, azaz a felhasználó saját kedve és becsülete szerint adhat hozzá elvégzettként különböző mozgásos feladatokat. Ezt a szoftver továbbfejlesztése esetén úgy lehetne figyelni, hogy az alkalmazás **elemezné a mobilkészüléken különböző alkalmazásokon keresztül a mozgást**. Például az Apple okoskészülékein az Egészség alkalmazáson belül található különböző mozgásos értékek alapján ítélné meg a HeBLE alkalmazás, hogy teljesítette-e a felhasználó a feladatokat. Ilyen alkalmazás megtalálható a különböző Android operációs rendszerrel felszerelt

okoseszközökön is. Samsung esetében a Samsung Health, de már maga a Google is rendelkezik egy hasonló alkalmazással, a Google Fit személyében.

A későbbiekben egy **új role** is bele épülhetne, ami pedig az **edző** lehetne. Neki nyilván a partnerei figyelése és progresszióinak nyomon követése mellett egy hálózatba bele kerülhetne, ami esetleg munkakeresési lehetőséget kínálna valamennyi hozzáértőnek. Természetesen ő is edzhet az alkalmazást használva, de hozzáértése miatt, akár nehezebb feladatokat is kaphat.

Végző, de nem utolsósorban továbbfejlesztési lehetőség rejlik abban is, hogy az webalkalmazásunkon rendelkezzen **saját online bolttal**, amelyben a felhasználók vásárolhatnak, illetve a **szintezés során különféle kedvezményekhez** juthatnának hozzá.

VIII. Irodalomjegyzék

A felhasznált irodalomjegyzék az alábbiakban tekinthető meg. Itt összegyűjtöttük mindent, amit felhasználtunk a projektünk elkészítéséhez. Minden csoporttag által felhasznált jegyzett külön-külön, névvel ellátva, keltezés szerint megjelölve. Fontos kihangsúlyoznunk, hogy minden, ami az oldalon megtalálható az csak localhoston keresztül érhető el és nincs semmi se publikálva. A képekre a Google böngészőjébe kerestünk rá.

Fodor Gergő által felhasználtak – a frontendhez:

<https://fontawesome.com/>

<https://www.w3schools.com/>

<https://angular.dev/>

<https://www.remove.bg/hu>

<https://imagecolorpicker.com/>

<https://stackoverflow.com/questions/22150006/align-two-divs-horizontally-side-by-side-center-to-the-page-using-bootstrap-css>

<https://stackoverflow.com/questions/23695090/spin-or-rotate-an-image-on-hover>

<https://stackoverflow.com/questions/78206018/how-to-scroll-to-top-on-route-change-in-angular-with-providerrouter/78206019#78206019>

<https://www.youtube.com/watch?v=15ovWuJ-Tcw>

https://www.youtube.com/watch?v=Am_DNrAlMMQ

<https://www.youtube.com/watch?v=gO8rL5iXk-Y>

<https://www.youtube.com/watch?v=7UREfYJCsCU>

2025.03.30

https://www.freepik.com/free-vector/neon-light-tunnel-black-background_370757410.htm#fromView=search&page=1&position=22&uuid=43745f86-c779-4793-943e-47527ac7b4fe&query=rgb+light+rod – alkotó: upklyak, Freepik

https://www.freepik.com/free-vector/siren-icon_201876770.htm#fromView=search&page=1&position=9&uuid=6a1e9699-e2d4-46da-baf3-6c3f3f835278&query=warning+sign – alkotó: xadartstudio, Freepik

2025.03.29. 16:35

https://www.freepik.com/free-psd/delicious-pile-peanuts-ready-be-enjoyed_405497074.htm#fromView=search&page=1&position=3&uuid=ad8e08c1-e47a-4fb5-8024-db40adc2e7f5&query=peanuts – alkotó: tohamina, Freepik

https://www.freepik.com/free-photo/close-up-fresh-kiwi-fruits-white-surface_2944472.htm#fromView=search&page=1&position=2&uuid=a15859db-1926-4e67-8bd9-9a6cbe0a51eb&query=kiwi+free+photos – alkotó: Freepik

https://www.freepik.com/free-photo/slice-ripe-lemon-citrus-fruit-white_10181238.htm#fromView=search&page=1&position=0&uuid=4ddc9668-12a0-4277-8a6f-55ece4759b64&query=Citrom – alkotó: standret, Freepik

https://www.freepik.com/free-vector/bright-yellow-sun-silhouette_136881498.htm#fromView=search&page=1&position=23&uuid=c290e65c-9e7e-4a2e-aed7-0a328d134789&query=Sun+cartoon – alkotó: gstudioimagen, Freepik

https://www.freepik.com/free-vector/coffee-cup-tan-colour_63885400.htm#fromView=search&page=1&position=8&uuid=ea37f23a-957e-4554-84aa-e39389014251&query=cup+coffee+drink – alkotó: juicy_fish, Freepik

https://www.freepik.com/free-vector/isolated-milk-box-cartoon-style_30155555.htm#fromView=search&page=1&position=11&uuid=42eba0f2-e137-4d75-9a01-a0cdf03f1eb7&query=milk+carton – alkotó: brgfx, Freepik

https://www.freepik.com/free-vector/soda-can-concept-illustration_138216643.htm#fromView=search&page=1&position=37&uuid=86ce5d18-fe32-4466-ac62-f761138742bd&query=cola+sugar+drink – alkotó: storyset, Freepik

https://www.freepik.com/free-vector/people-water-big-flat-color-set-with-young-woman-fitness-suit-drinking-water-from-bottle-isolated-vector-illustration_37421559.htm#fromView=search&page=1&position=5&uuid=8a2d9304-

[1dd4-43f5-883c-fa5fc4b73c1a&query=body+water+percentage+](https://www.freepik.com/free-vector/body+water+percentage) alkotó: macrovector,
Freepik

https://www.freepik.com/free-photo/grapes-strawberries-pineapple-kiwi-apricot-banana-whole-pineapple_3630863.htm#fromView=search&page=1&position=0&uuid=80b24abe-b1bc-4710-8dc6-811210a84dc2&query=variety+of+fruits - alkotó: Freepik

https://www.freepik.com/free-vector/stretching-exercises-concept-illustration_24487813.htm#fromView=search&page=1&position=8&uuid=7d45e3f5-bd88-4641-aa9a-c53297d248eb&query=warmup – alkotó: storyset, Freepik

https://www.freepik.com/free-vector/insomnia-concept-with-man_9908275.htm#fromView=search&page=1&position=13&uuid=ef1889d1-707d-4a36-b801-c926692641dc&query=sleep+sheep+counting – alkotó: Freepik

https://www.freepik.com/free-vector/woman-suffering-from-insomnia-flat-vector-illustration-depressed-sad-female-person-lying-bed-with-open-eyes-unable-sleep-night-sleep-disorder-fatigue-sleepless-concept_28480935.htm#fromView=search&page=7&position=43&uuid=c067de13-c906-4f0d-b82f-59ae83a1855a&query=sleeping+ alkotó: pch.vector, Freepik

https://www.freepik.com/free-vector/crescent-moon-stars-clear-sky-background_9727976.htm#fromView=search&page=4&position=42&uuid=4550a7c5-8a83-4346-a8d1-03ba670d52fc&query=moon – alkotó: starline, Freepik

https://www.freepik.com/free-photo/top-view-perfectly-ordered-fitness-items_40483717.htm - alkotó: Freepik

2025.03.16 11:38

<https://www.capetansport.hu/edzestipusok-es-hatakonysaguk>

https://www.freepik.com/free-vector/flat-design-bike-race-illustration_28554008.htm -
alkotó: Freepik

https://www.freepik.com/free-vector/woman-doing-yoga-her-house_8132282.htm#fromView=search&page=1&position=5&uuid=03ff6a90-2503-48ab-a702-ecc300b4f206&query=Home+Cardio+Workout alkotó: Freepik

https://www.freepik.com/free-vector/online-yoga-class-concept_8246642.htm#fromView=search&page=1&position=13&uuid=10dcb60d-88a6-4a41-ad94-336ce03e4d61&query=Home+Cardio+Workout alkotó: Freepik

https://www.freepik.com/free-vector/hand-drawn-online-sport-classes-concept-illustrated_12242146.htm#fromView=search&page=1&position=0&uuid=03ff6a90-2503-48ab-a702-ecc300b4f206&query=Home+Cardio+Workout alkotó: Freepik

https://www.freepik.com/free-vector/online-sport-classes-woman-laptop_12260513.htm#fromView=search&page=1&position=0&uuid=1d928f4d-bb5f-4f47-900c-a03913168b85&query=Home+Pilates+Workout alkotó: Freepik

2025.03.11 23:39

https://www.freepik.com/free-vector/golden-silver-bronze-metallic-trophy-cup-set-isolated-vector-illustration_1158422.htm - alkotó: Freepik

2025.02.08 11:47

https://www.freepik.com/free-vector/bottle-water-concept-illustration_8476696.htm#fromView=keyword&page=1&position=3&uuid=c66f60e5-8b9e-4878-9491-4e6b1de73913&query=2d+Bottle+Illustration – alkotó: storyset, Freepik

<https://www.simplesport.hu/hidratacio/>

<https://www.webbeteg.hu/cikkek/egeszsegedre/17512/hidratacio>

<https://www.webbeteg.hu/cikkek/egeszseges/21708/tej-es-tejtermekfogyaszta>

<https://www.webbeteg.hu/cikkek/egeszseges/20181/tudomanyos-tenyek-a-kavefogyaszta>

https://www.hazipatika.com/taplalkozas/egeszseg_es_asztronomia/cikkek/a_cukros_uditok_sokfele_betegseget_okozhatnak

2025.02.05 11:51

https://www.freepik.com/free-vector/gradient-yoga-poses-collection_25810928.htm#fromView=search&page=1&position=18&uuid=8dfcac64-4232-4a5c-835f-cf9f9360c938&query=warmup - alkotó: pikisuperstar, Freepik

<https://www.intenset.hu/blog/nyujtas-es-bemelegites-gyakorlatok/>

https://shopbuilder.hu/bemelegito-gyakorlatok-a20526?srsId=AfmBOoo3-wKKql8W-HZpk5wbwlCLjy0E_ipwBiCv6QyJTd2wJjTTBJXD

<https://gymbeam.hu/blog/nyujtas-bemelegites-es-levezetes-valoban-fontosak/>

<https://shopbuilder.hu/nyujtas-a1724?srsId=AfmBOooL9Kw4xl36-5XRBSfAZKg1nXCBz6n2OLpYN1dIhEPu229KvDxf>

<https://gymbeam.hu/blog/hogyan-lehet-javitani-az-alvas-minosegen-es-hogyan-befolyasolja-az-az-egeszseget-es-az-izomnovelkedes/>

<https://egeszsegvonal.gov.hu/maradj-egeszseges/egeszseges-alvasi-szokasok.html>

<https://biotechusa.hu/blog/eletmod/az-alvas-jelentosege-regeneracios-szemponthoz/>

2025.02.04 20:24

<https://biotechusa.hu/blog/taplalek-kiegészítő-tippek/vitaminok/>

<https://biotechusa.hu/blog/taplalek-kiegészítő-tippek/a-feherjeturmix-legfontosabb-hatasai/>

<https://eletmodvaltasmost.com/mi-a-taplalekkiegészítő-es-miert-van-ra-szükségünk/>

<https://benu.hu/cikk/melyek-a-legjobb-feherjeforrások-mutatjuk>

<https://semmelweis.hu/egeszsegfejlesztés/2022/02/24/vitaminok-etrend-kiegészítők-az-egeszsegugyi-szakemberre-hallgassunk-ne-a-reklamokra-interjú-dr-somogyi-orsolyával-1-resz/>

2025.01.31 13:54

https://www.freepik.com/free-vector/gradient-home-gym-with-different-elements_11827840.htm#fromView=search&page=1&position=0&uuid=36280811-d22a-4015-8326-b746dd68a47c&query=workout+place - alkotó: Freepik

2025.01.28 23:01

https://commons.wikimedia.org/wiki/File:Font_Awesome_5_solid_quote-right.svg

https://commons.wikimedia.org/wiki/File:Font_Awesome_5_solid_quote-left.svg

(Wikipédia, Font Awesome – módosítás: fehér háttér eltüntetve)

Sedlák Patrik által felhasználtak – a backendhez:

<https://fakerjs.dev/>

<https://www.npmjs.com/package/@faker-js/faker>

<https://stackoverflow.com/questions/32010910/faker-js-random-number-between-2-values>

<https://en.wikipedia.org/wiki/Bcrypt>

<https://www.npmjs.com/package/bcrypt>

<https://www.npmjs.com/package/bcryptjs>

https://en.wikipedia.org/wiki/Cross-origin_resource_sharing

<https://www.npmjs.com/package/cors>

<https://www.dotenv.org>

<https://www.npmjs.com/package/dotenv>

<https://expressjs.com>

<https://devdocs.io/express/>

<https://www.geeksforgeeks.org/express-js/>

<https://www.npmjs.com/package/express-session>

<https://expressjs.com/en/resources/middleware/session.html>

<https://www.npmjs.com/package/jsonwebtoken>

<https://www.geeksforgeeks.org/json-web-token-jwt>

<https://www.npmjs.com/package/mariadb>

<https://www.npmjs.com/package/nodemon>

<https://sequelize.org>

<https://www.npmjs.com/package/sequelize>

<https://docs.nestjs.com/recipes/sql-sequelize>

